Bazele Inteligenţei Artificiale

Prof. Dr. Călin-Adrian POPA

Cursul 3

14 Octombrie 2025

3 Decision trees. Ensemble methods
3.1 Decision trees

- decision trees are widely used models for classification and regression tasks
- essentially, they learn a hierarchy of *if/else questions*, leading to a *decision*
- these questions are similar to the questions we might ask in a game of *20 Questions*
- imagine we want to distinguish between the following four animals: bears, hawks, penguins, and dolphins
- our goal is to get to the right answer by asking as few if/else questions as possible

3.1 Decision trees

- we might start off by asking whether the animal has feathers, a question that narrows down our possible animals to just two
- if the answer is “yes”, we can ask another question that could help us distinguish between hawks and penguins
- for example, we could ask whether the animal can fly
- if the animal doesn’t have feathers, our possible animal choices are dolphins and bears, and we will need to ask a question to distinguish between these two animals – for example, asking whether the animal has fins
- this series of questions can be expressed as a decision tree, as shown in Figure 1

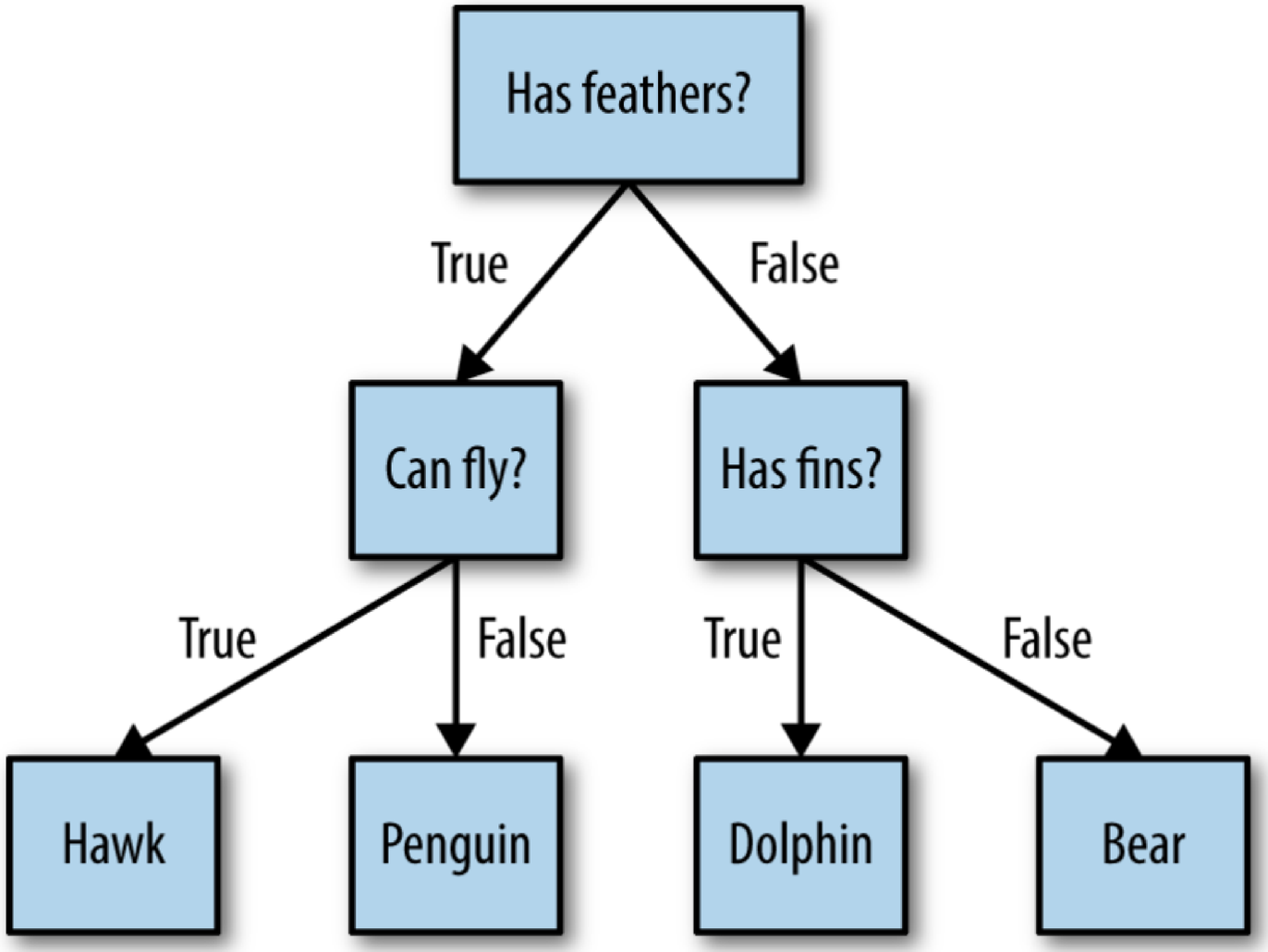


Figure 1: A decision tree to distinguish among several animals.

3.1 Decision trees

- in this illustration, each node in the tree either represents a question or a terminal node (also called a *leaf*) that contains the answer
- the edges connect the answers to a question with the next question we would ask
- in machine learning terms, we built a model to distinguish between four classes of animals (hawks, penguins, dolphins, and bears) using the three features “has feathers”, “can fly”, and “has fins”
- instead of building these models by hand, we can learn them from data

3.1 Decision trees

- let's go through the process of building a decision tree for the 2D *binary classification* dataset shown in Figure 2
- the dataset consists of two half-moon shapes, with $q = 2$ classes, each class consisting of 50 data points
- we will refer to this dataset as `two_moons`

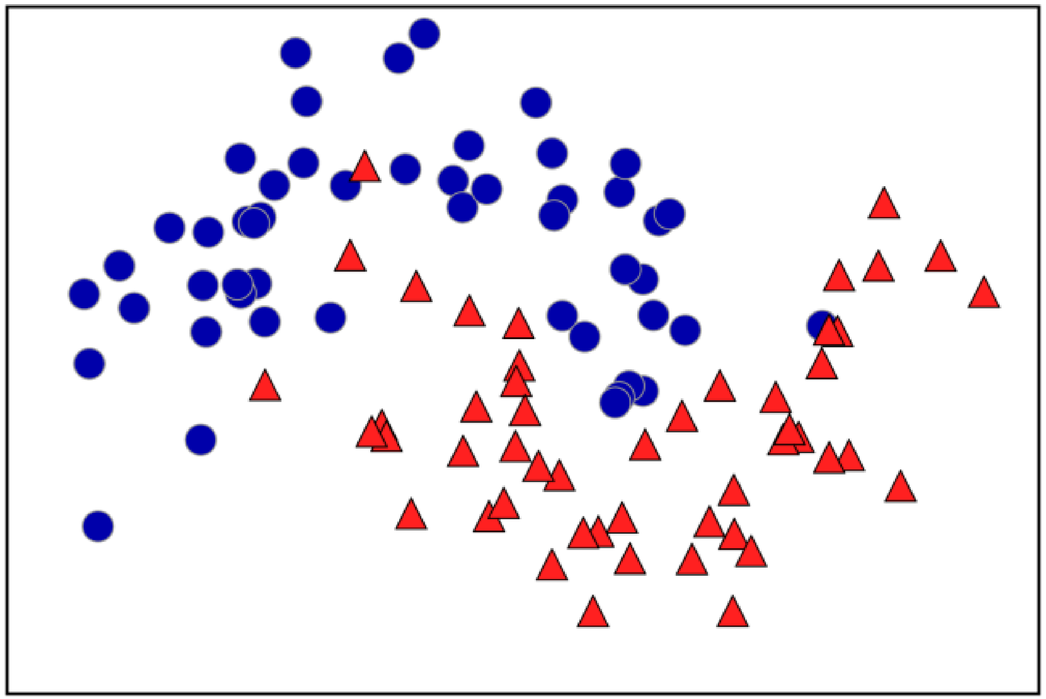


Figure 2: The `two_moons` dataset on which the decision tree will be built.

3.1 Decision trees

- learning a decision tree means learning the sequence of if/else questions that gets us to the true answer most quickly
- in the machine learning setting, these questions are called *tests* (not to be confused with the test set, which is the data we use to test to see how generalizable our model is)
- usually, data does not come in the form of binary yes/no features as in the animal example, but is instead represented as continuous features, such as in the 2D dataset shown in Figure 2
- the tests that are used on continuous data are of the form "Is feature x_j larger than value t?"

3.1 Decision trees

- to build a tree, the algorithm searches over all possible tests and finds the one that is most informative about the target variable
- Figure 3 shows the first test that is picked

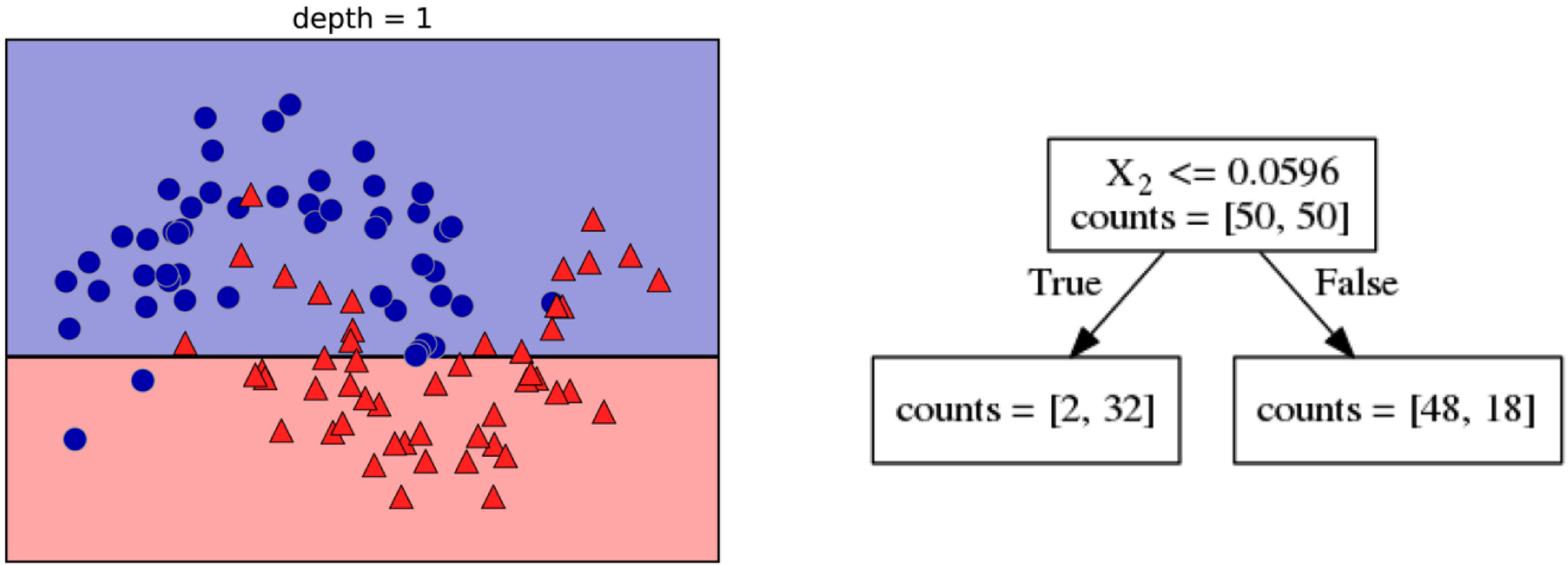
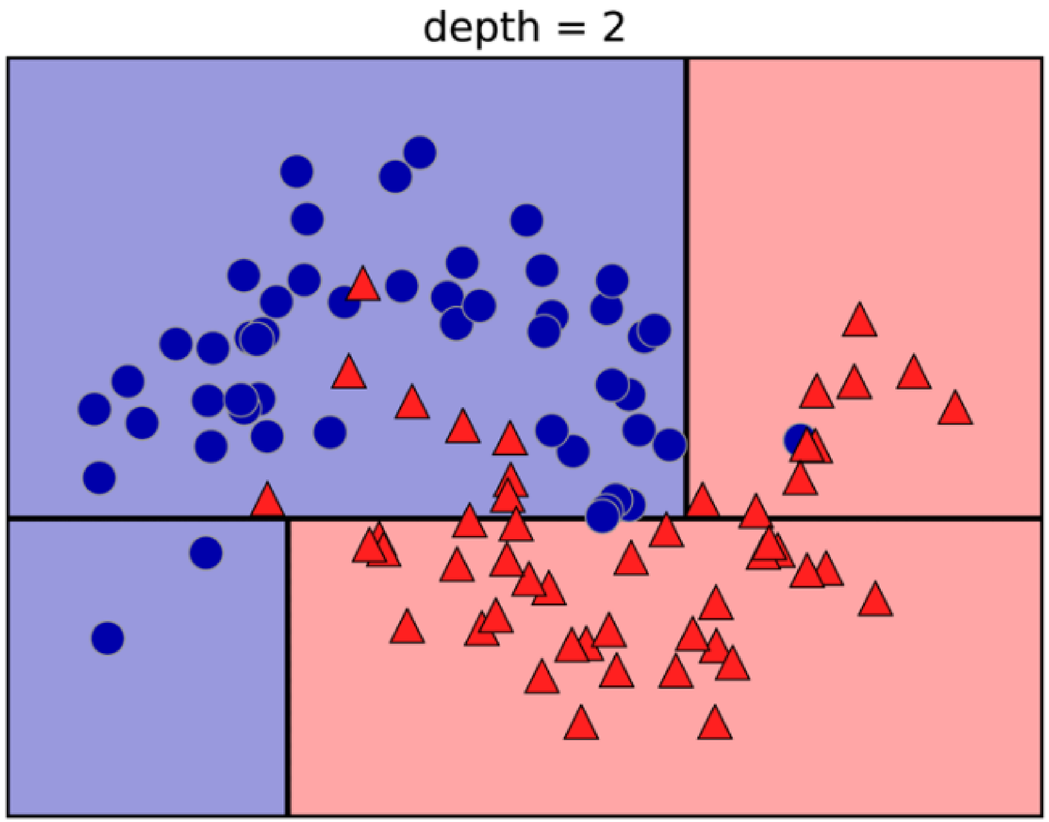


Figure 3: Decision boundary of tree with depth 1 (left) and corresponding decision tree (right).

3.1 Decision trees

- splitting the dataset vertically at $x_2 = 0.0596$ yields the most information; it best separates the points in class 1 from the points in class 2
- the top node, also called the *root*, represents the whole dataset, consisting of 50 points belonging to class 1 and 50 points belonging to class 2
- the split is done by testing whether $x_2 \leq 0.0596$, indicated by a black line
- if the test is true, a point is assigned to the left node, which contains 2 points belonging to class 1 and 32 points belonging to class 2
- otherwise, the point is assigned to the right node, which contains 48 points belonging to class 1 and 18 points belonging to class 2

- these two nodes correspond to the top and bottom regions shown in Figure 3
- even though the first split did a good job of separating the two classes, the bottom region still contains points belonging to class 1, and the top region still contains points belonging to class 2
- we can build a more accurate model by repeating the process of looking for the best test in both regions
- Figure 4 shows that the most informative next split for the left and the right region is based on x_1



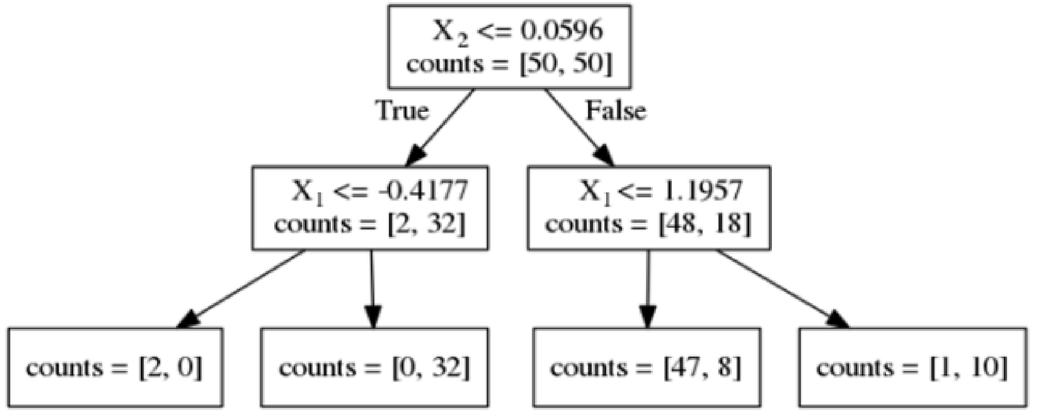


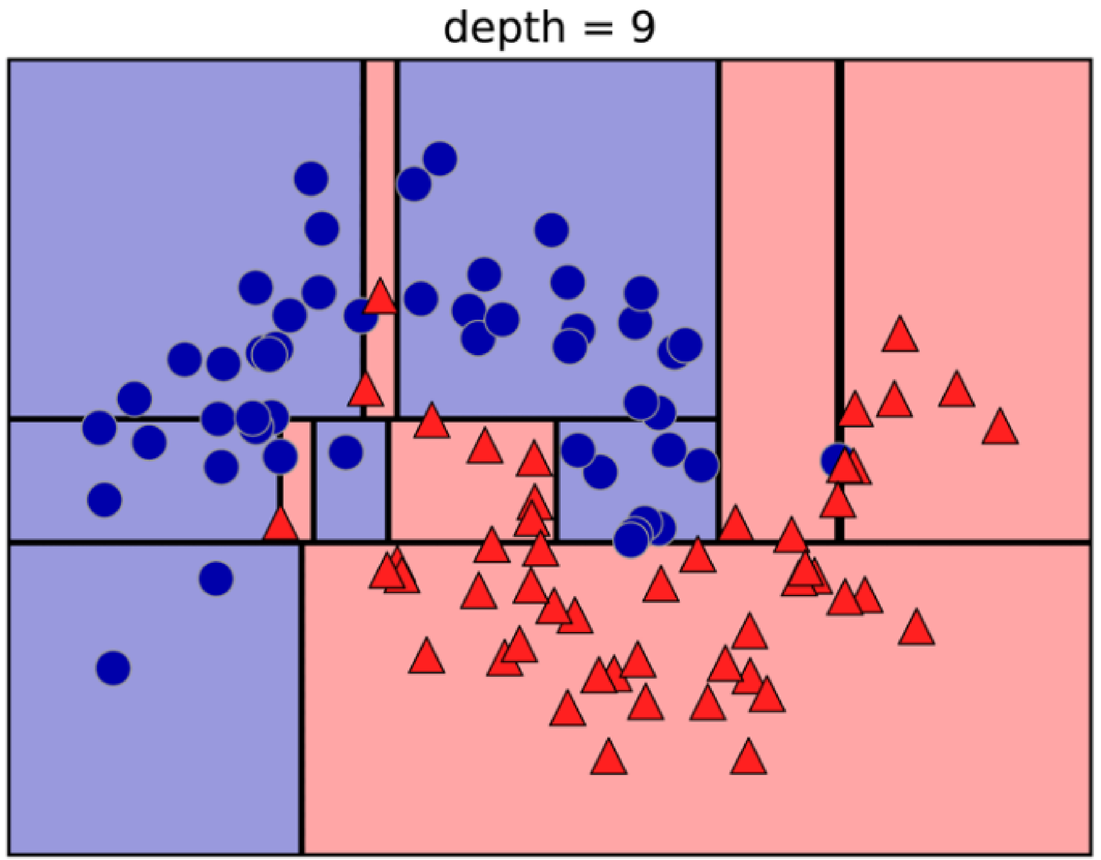
Figure 4: Decision boundary of tree with depth 2 (left) and corresponding decision tree (right).

3.1 Decision trees

- this recursive process yields a binary tree of decisions, with each node containing a test
- alternatively, we can think of each test as splitting the part of the data that is currently being considered along one axis
- this yields a view of the algorithm as building a hierarchical partition
- as each test concerns only a single feature, the regions in the resulting partition always have *axis-parallel boundaries*
- the recursive partitioning of the data is repeated until each region in the partition (each leaf in the decision tree) only contains a single target value (a single class or a single regression value)
- a leaf of the tree that contains data points that all share the same target value is called *pure*

3.1 Decision trees

- the final partitioning for this dataset is shown in Figure 5



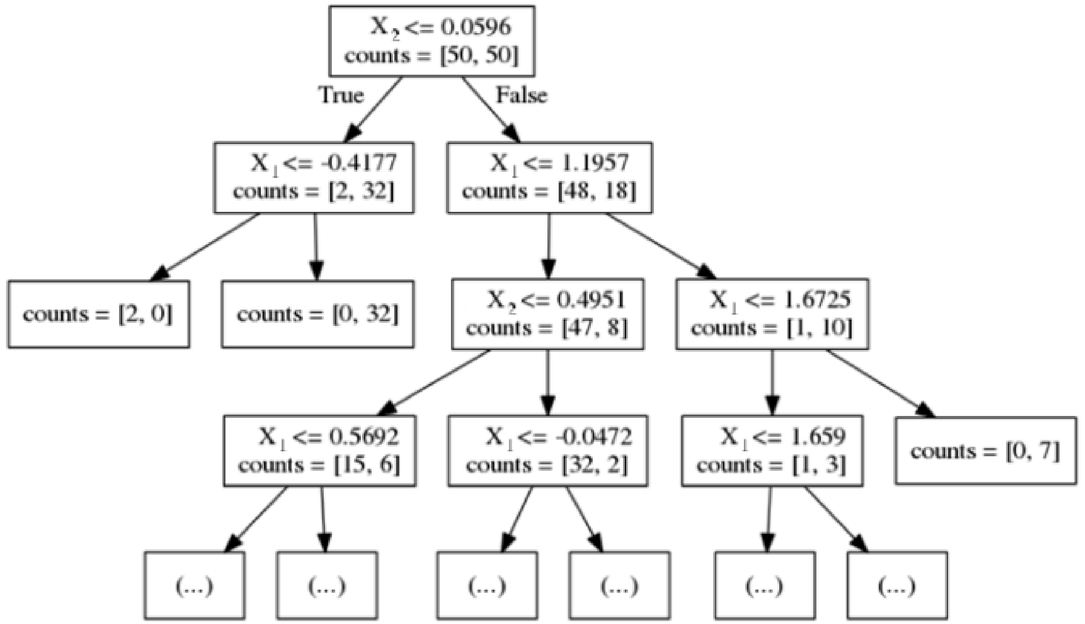


Figure 5: Decision boundary of tree with depth 9 (left) and part of the corresponding decision tree (right); the full tree is quite large and hard to visualize.

3.1 Decision trees

- a prediction on a new data point is made by checking which region of the partition of the feature space the point lies in, and then predicting the *majority* target (or the single target in the case of pure leaves) in that region
- the region can be found by traversing the tree from the root and going left or right, depending on whether the test is fulfilled or not
- it is also possible to use trees for *regression* tasks, using exactly the same technique
- to make a prediction, we traverse the tree based on the tests in each node and find the leaf the new data point falls into
- the output for the data point is the *mean* target of the training points in that leaf

3.1 Decision trees

- one of the many qualities of decision trees is that they require very little data preparation
- in fact, they don't require feature scaling or centering at all
- decision trees are intuitive, and their decisions are easy to interpret
- such models are often called *white box models*

3.1 Decision trees

- in contrast, as we will see, random forests or neural networks are generally considered *black box models*
- they make great predictions, and we can easily check the calculations that they performed to make these predictions; nevertheless, it is usually hard to explain in simple terms why the predictions were made
- for example, if a neural network says that a particular person appears in a picture, it is hard to know what contributed to this prediction: did the model recognize that person's eyes? their mouth? their nose? their shoes? or even the couch that they were sitting on?
- conversely, decision trees provide nice, simple classification rules that can even be applied manually, if need be

3.1 Decision trees

- but how are decision trees trained?
- one popular algorithm is the *Classification and Regression Tree (CART)* algorithm, also called "*growing*" *trees*
- the CART algorithm produces only *binary trees*: non-leaf nodes always have two children (i.e., questions only have yes/no answers)
- however, other algorithms, such as ID3, can produce decision trees with nodes that have more than two children

3.1 Decision trees

- we now discuss the process of building a regression tree
- roughly speaking, there are two steps:
  1. We divide the feature space – that is, the set of possible values for the input features $x_1, x_2, \ldots, x_d$ – into m distinct and non-overlapping regions, $R_1, R_2, \ldots, R_m$. In theory, the regions could have any shape. However, we choose to divide the feature space into high-dimensional rectangles, or *boxes*, for simplicity and for ease of interpretation of the resulting predictive model.
  2. For every example that falls into the region R_h, we make the same prediction, which is simply the mean of the response values for the training examples in R_h. The output for region h can be estimated as follows:

$$\hat{y}_{R_h} = \frac{\sum_{i=1}^{n} y^{(i)} 1_{\boldsymbol{x}^{(i)} \in R_h}}{\sum_{i=1}^{n} 1_{\boldsymbol{x}^{(i)} \in R_h}},$$

  where $\hat{y}_{R_h}$ is the mean response for the training examples within R_h.

- the regions $R_1, R_2, \ldots, R_m$ represent the *terminal nodes* or *leaves* of the tree

3.1 Decision trees

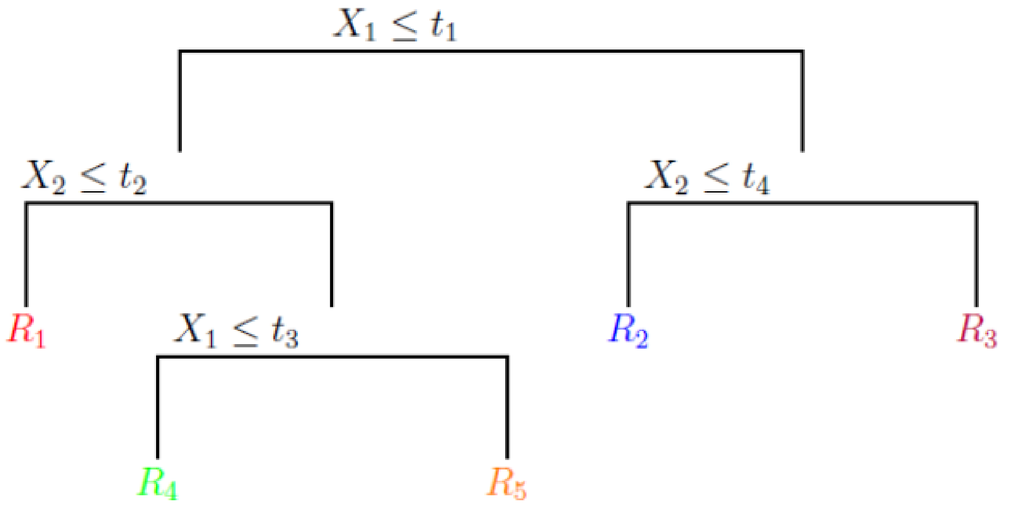
- decision trees are typically drawn *upside down*, in the sense that the leaves are at the bottom of the tree
- the points along the tree where the feature space is split are the *internal nodes*
- the segments of the trees that connect the nodes are the *branches*
- the regression tree can be defined by:

$$f(\boldsymbol{x}; \boldsymbol{w}) = \sum_{h=1}^{m} \hat{y}_{R_h} 1_{\boldsymbol{x} \in R_h},$$

  where R_h is the region specified by the hth leaf node, $\hat{y}_{R_h}$ is the predicted output for that node, $\boldsymbol{w} = \{(R_h, \hat{y}_{R_h}) | h = 1, 2, \ldots, m\}$, and m is the number of nodes
- the regions themselves are defined by the feature dimensions that are used in each split, and the corresponding thresholds, on the path from the root to the leaf

3.1 Decision trees

- for example, in Figure 6, we have $R_1 = \{\boldsymbol{x} | x_1 \leq t_1, x_2 \leq t_2\}$, $R_2 = \{\boldsymbol{x} | x_1 > t_1, x_2 \leq t_4\}$, etc.
- for categorical inputs, we can define the splits based on comparing feature x_j to each of the possible values for that feature, rather than comparing to a numeric threshold



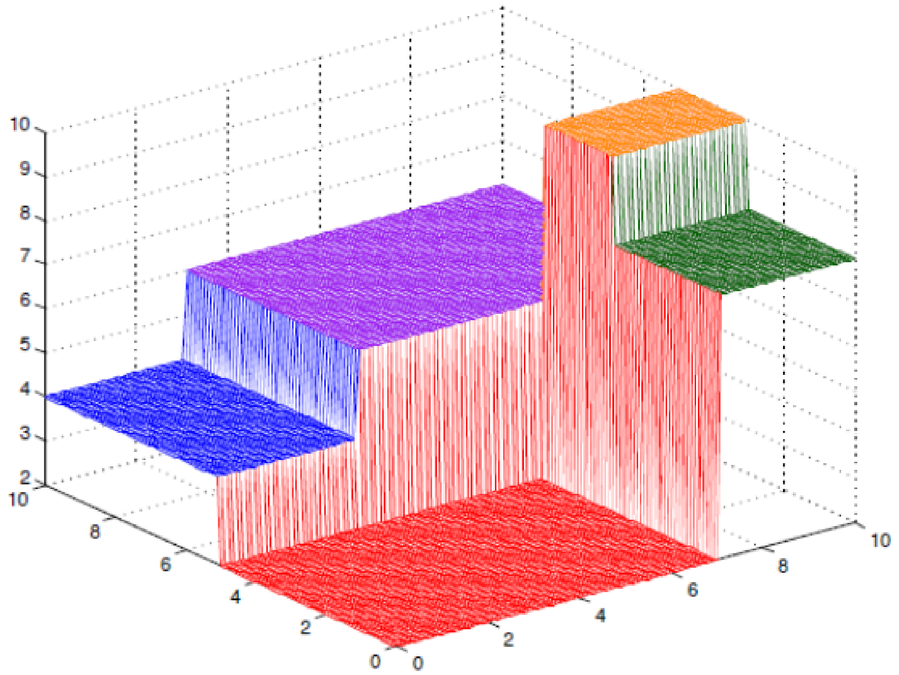


Figure 6: A simple general regression tree with two inputs.

- how do we construct the regions $R_1, R_2, \ldots, R_m$?
- the goal is to find boxes $R_1, R_2, \ldots, R_m$ that minimize the loss, given by:

$$\mathcal{L}(\boldsymbol{w}) = \sum_{i=1}^{n} l^{(i)}(y^{(i)}, f(\boldsymbol{x}^{(i)}; \boldsymbol{w})) = \sum_{h=1}^{m} \sum_{i:\, \boldsymbol{x}^{(i)} \in R_h} l^{(i)}(y^{(i)}, \hat{y}_{R_h}).$$

- unfortunately, this is not differentiable, because of the need to learn the discrete tree structure
- also, finding the optimal tree is known to be an *NP-complete* problem: it requires $\mathcal{O}(\exp(n))$ time, making it computationally infeasible to consider every possible partition of the feature space into m boxes, even for small training sets

3.1 Decision trees

- for this reason, we take a *top-down*, *greedy* approach that is known as *recursive binary splitting*, in which we iteratively grow the tree one node at a time
- the approach is *top-down* because it begins at the top of the tree (at which point all examples belong to a single region) and then successively splits the feature space; each split is indicated via two new branches further down on the tree
- it is *greedy* because, at each step of the tree-building process, the *best* split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step

3.1 Decision trees

- in order to perform recursive binary splitting, we first select the feature x_j and the threshold t such that splitting the feature space into the regions $\{\boldsymbol{x}|x_j \leq t\}$ and $\{\boldsymbol{x}|x_j > t\}$ leads to the *greatest possible reduction to the loss*
- the notation $\{\boldsymbol{x}|x_j \leq t\}$ means the region of feature space in which x_j takes on a value less than, or equal to, t
- for categorical features, we use $\{\boldsymbol{x}|x_j = t\}$ and $\{\boldsymbol{x}|x_j \neq t\}$
- that is, we consider all features $x_1, \ldots, x_d$, and all possible values of the threshold t for each of the features, and then choose the feature and threshold such that the resulting tree has the lowest loss

3.1 Decision trees

- in greater detail, suppose we are at node h, and let $\mathcal{D}_h = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}$ be the set of examples that reach this node
- for any j and t, let $\mathcal{D}_h^{left}(j,t)$ and $\mathcal{D}_h^{right}(j,t)$ be a partition of these examples: $\mathcal{D}_h^{left}(j,t) = \{(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h | x_j^{(i)} \leq t\}$, $\mathcal{D}_h^{right}(j,t) = \{(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h | x_j^{(i)} > t\}$
- for categorical features, we use $\mathcal{D}_h^{left}(j,t) = \{(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h | x_j^{(i)} = t\}$, $\mathcal{D}_h^{right}(j,t) = \{(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h | x_j^{(i)} \neq t\}$
- we choose the best feature j to split on, and the best threshold for that feature, t, in order to minimize the following loss:

$$\mathcal{L}(j,t) = \frac{|\mathcal{D}_h^{left}(j,t)|}{|\mathcal{D}_h|}\mathcal{L}(\mathcal{D}_h^{left}(j,t)) + \frac{|\mathcal{D}_h^{right}(j,t)|}{|\mathcal{D}_h|}\mathcal{L}(\mathcal{D}_h^{right}(j,t)), \tag{1}$$

i.e.,

$$(j_h, t_h) = \underset{(j \in \{1,2,\ldots,d\}, t \in \mathcal{T}_j)}{\operatorname{argmin}} \mathcal{L}(j,t).$$

3.1 Decision trees

- the set of possible thresholds $\mathcal{T}_j$ for feature j can be obtained by sorting the unique values of $\{x_j^{(i)}\}$
- for example, if feature j has the values $\{4.5, -12, 72, -12\}$, then we set $\mathcal{T}_j = \{-12, 4.5, 72\}$
- in the case of categorical inputs, suppose feature j has K_j possible values
- then we check if the feature is equal to each of those values or not
- this defines a set of K_j possible binary splits

3.1 Decision trees

- we now discuss the loss function $\mathcal{L}(\mathcal{D})$; for regression, we can use the mean squared error:

$$\mathcal{L}(\mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i:\,(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}} (y^{(i)} - \hat{y}_{\mathcal{D}})^2,$$

where:

$$\hat{y}_{\mathcal{D}} = \frac{\sum_{i=1}^{n} y^{(i)} 1_{(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}}}{\sum_{i=1}^{n} 1_{(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}}} = \frac{1}{|\mathcal{D}|} \sum_{i:\,(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}} y^{(i)}.$$

- now, (1) becomes:

$$\mathcal{L}(j, t) = \frac{1}{|\mathcal{D}_h|} \left(\sum_{i:\,(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h^{left}(j,t)} (y^{(i)} - \hat{y}_{\mathcal{D}_h^{left}(j,t)})^2 + \sum_{i:\,(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h^{right}(j,t)} (y^{(i)} - \hat{y}_{\mathcal{D}_h^{right}(j,t)})^2 \right). \quad (2)$$

- finding the values of j and t that minimize (2) can be done quite quickly, especially when the number of features d is not too large

- next, we repeat the process, looking for the best feature and best threshold in order to split the data further so as to minimize the loss within each of the *resulting regions*
- however, this time, instead of splitting the entire feature space, we split one of the two previously identified regions
- we now have three regions; again, we look to split one of these three regions further, so as to minimize the loss
- the process continues until a *stopping criterion* is reached; for instance, we may continue until the tree reaches the maximum depth, or if it cannot find a split that will reduce the loss
- once the regions $R_1, R_2, \ldots, R_m$ have been created, we predict the response for a given test example using the mean of the training examples in the region which that test example belongs to

3.1 Decision trees

- a *classification tree* is very similar to a regression tree, except that it is used to predict a qualitative response, rather than a quantitative one
- recall that, for a regression tree, the predicted response for an example is given by the mean response of the training examples that belong to the same terminal node (leaf)
- in contrast, for a classification tree, we predict that each example belongs to the *most commonly occurring class* of training examples in the region which it belongs to
- in interpreting the results of a classification tree, we are often interested not only in the class prediction corresponding to a particular terminal node region, but also in the *class proportions* among the training examples that fall into that region

3.1 Decision trees

- the task of growing a classification tree is quite similar to the task of growing a regression tree
- just as in the regression setting, we use recursive binary splitting to grow a classification tree
- however, in the classification setting, mean squared error cannot be used as a criterion for making the binary splits
- a natural alternative to mean squared error is the *classification error rate*

- since we plan to assign an example in a given region to the *most commonly occurring class* of training examples in that region, the classification error rate is simply the fraction of the training examples in that region that do not belong to the most common class:

$$E_h = 1 - \max_k \hat{p}_{hk},$$

where $\hat{p}_{hk}$ represents the proportion of training examples in the hth region that are from the kth class:

$$\hat{p}_{hk} = \frac{1}{|\mathcal{D}_h|} \sum_{i:\,(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h} 1_{y^{(i)}=k}.$$

- however, it turns out that classification error is *not sufficiently sensitive* for tree-growing, and in practice two other measures are preferable

- the *Gini index* is defined by:

$$G_h = \sum_{k=1}^{q} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \sum_{k=1}^{q} \hat{p}_{hk} - \sum_{k=1}^{q} \hat{p}_{hk}^2 = 1 - \sum_{k=1}^{q} \hat{p}_{hk}^2,$$

  a measure of total variance across the q classes
- it is not hard to see that the Gini index takes on a small value if all of the $\hat{p}_{hk}$'s are close to zero or one
- for this reason, the Gini index is referred to as a measure of node *purity* – a small value indicates that a node contains predominantly examples from a single class
- it can also be seen as the *expected error rate*
- to see this, note that $\hat{p}_{hk}$ is the probability a random entry in the leaf belongs to class k, and $1 - \hat{p}_{hk}$ is the probability it would be misclassified

- an alternative to the Gini index is *entropy*, given by:

$$H_h = H(\hat{\boldsymbol{p}}_h) = -\sum_{k=1}^{q} \hat{p}_{hk} \log \hat{p}_{hk}.$$

- one can show that the entropy will take on a value near zero if the $\hat{p}_{hk}$'s are all near zero or near one
- therefore, like the Gini index, the entropy will take on a small value if the hth node has high purity
- at the limit, a node that is *pure* (i.e., only has examples of one class) will have 0 entropy and 0 Gini index
- in fact, it turns out that the Gini index and the entropy are quite similar numerically
- when building a classification tree, either the Gini index or the entropy are typically used to evaluate the quality of a particular split, since these two approaches are *more sensitive* to node purity than is the classification error rate

3.1 Decision trees

- if we let the tree become deep enough, it can achieve 0 error on the training set (assuming no label noise), by partioning the input space into sufficiently small regions where the output is constant
- however, this will typically result in overfitting, leading to poor test set performance
- this is because the resulting tree might be too complex
- a smaller tree with fewer splits (that is, fewer regions $R_1, R_2, \ldots, R_m$) might lead to lower variance and better interpretation, at the cost of a little bias
- to avoid overfitting the training data, we need to restrict the decision tree's freedom during training; as we already know, this is called *regularization*

3.1 Decision trees

- there are two main approaches for performing decision tree regularization
- the first is to *stop* the tree growing process according to some heuristic, such as having too few examples at a node, or reaching a maximum depth
- the second approach is to grow the tree to its maximum depth, where no more splits are possible, and then to *prune* it back, by merging split subtrees back into their parent
- this can partially overcome the greedy nature of top-down tree growing
- however, forward growing and backward pruning is slower than the greedy top-down approach

- a node whose children are all leaf nodes is considered unnecessary if the purity improvement it provides is not *statistically significant*
- standard statistical tests, such as the χ^2 test (chi-squared test), are used to estimate the probability that the improvement is purely the result of chance (i.e., there is no underlying pattern, which is called the *null hypothesis*)
- if this probability is higher than a given threshold (typically 5%, controlled by a hyperparameter), then the node is considered unnecessary and its children are deleted
- the pruning continues until all unnecessary nodes have been pruned

3.1 Decision trees

- decision trees for regression and classification have a number of advantages:
  - they are easy to interpret and to explain to people; in fact, they are even easier to explain than linear regression
  - some people believe that decision trees more closely mirror human decision-making than do the regression and classification approaches seen in previous lectures
  - trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small)
  - trees can easily handle qualitative features without the need to create dummy variables
  - they can easily handle mixed discrete and continuous inputs
  - they are insensitive to monotone transformations of the inputs (because the split points are based on ranking the data points), so there is no need to normalize or scale the data
  - they perform automatic variable selection
  - they are relatively robust to outliers
  - they are fast to fit, and scale well to large datasets

- however, tree models also have some disadvantages:
  - trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches seen in previous lectures. This is in part due to the greedy nature of the tree construction algorithm.
  - trees can be very *non-robust* or *unstable*. In other words, small changes to the input data can have large effects on the structure of the tree, due to the hierarchical nature of the tree-growing process, causing errors at the top to affect the rest of the tree. Omitting even a single data point from the training set can result in a dramatically different decision surface, due to the use of axis parallel splits. Omitting features can also cause instability.
- nonetheless, by aggregating many decision trees, using methods like voting, bagging, random forests, and boosting, the predictive performance of trees can be substantially improved; we introduce these concepts next

3.2 Voting

- suppose we pose a complex question to thousands of random people, then aggregate their answers
- in many cases, we will find that this aggregated answer is better than an expert's answer; this is called the *wisdom of the crowd*
- similarly, if we aggregate the predictions of a group of predictors (such as classifiers or regressors), we will often get better predictions than with the best individual predictor
- a group of predictors is called an *ensemble*; thus, this technique is called *ensemble learning*, and an ensemble learning algorithm is called an *ensemble method*
- the ensemble will have similar bias to the base models, but lower variance, generally resulting in improved overall performance

3.2 Voting

- suppose we have trained a few classifiers, each one achieving about 80% accuracy
- we may have a logistic regression classifier, an SVM classifier, a random forest classifier, a k-nearest neighbors classifier, and perhaps a few more (see Figure 7)
- a very simple way to create an even better classifier is to aggregate the predictions of each classifier and predict the class that gets the most votes
- this majority vote classifier is called a *hard voting* classifier (see Figure 8)

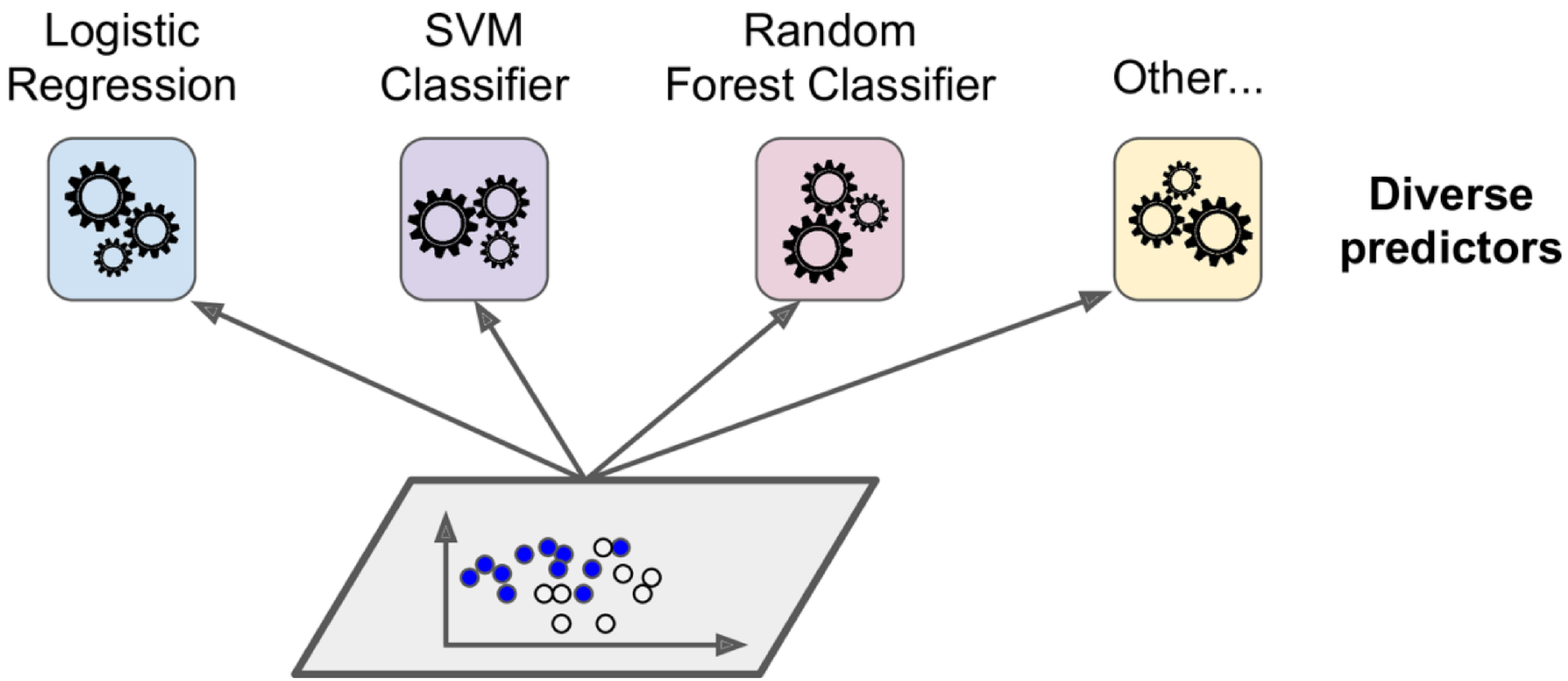


Figure 7: Training diverse classifiers.

3.2 Voting

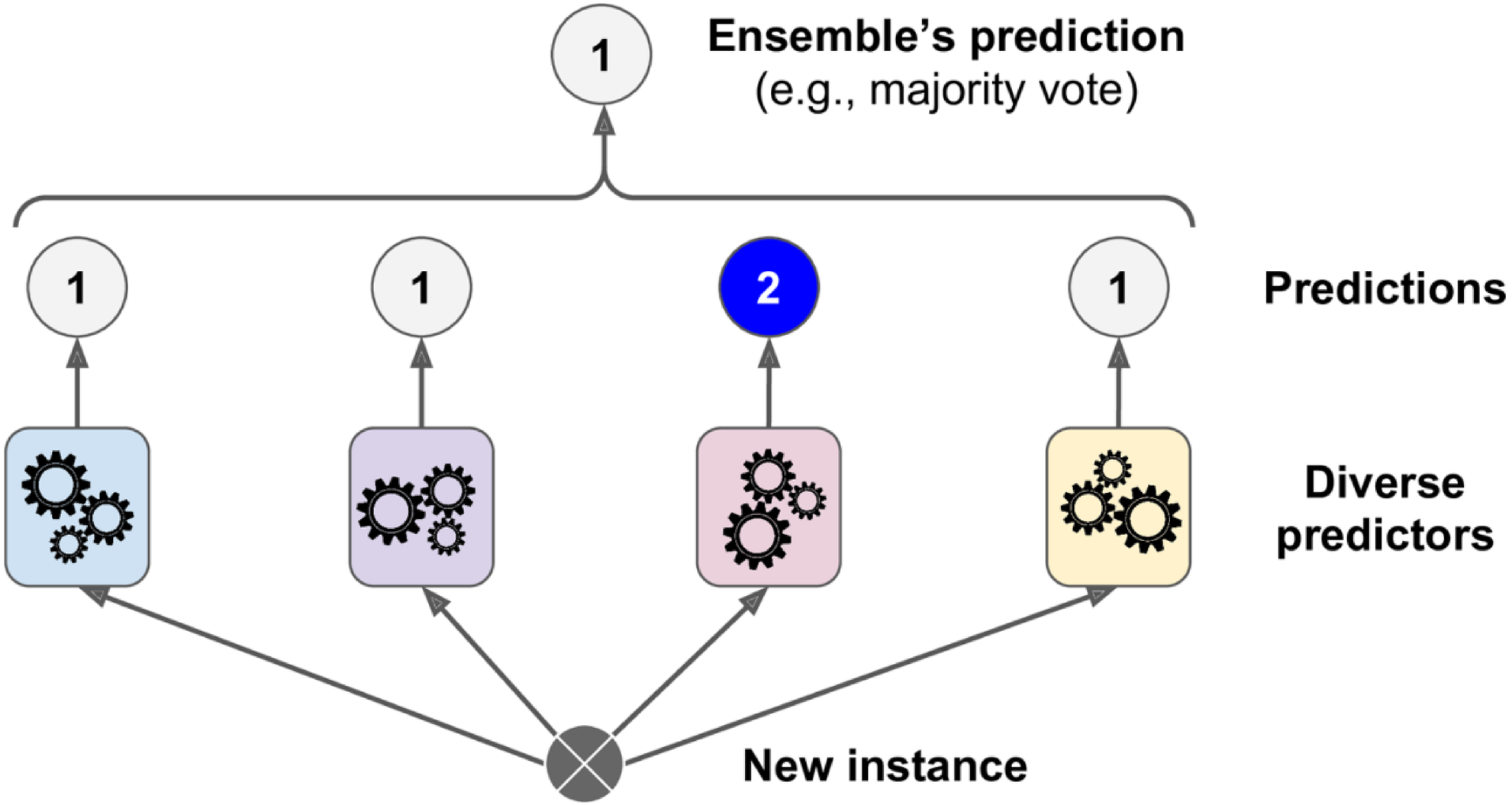


Figure 8: Hard voting classifier predictions.

3.2 Voting

- somewhat surprisingly, this voting classifier often achieves a higher accuracy than the best classifier in the ensemble
- in fact, even if each classifier is a *weak learner* (meaning it does only slightly better than random guessing), the ensemble can still be a *strong learner* (achieving high accuracy), provided there are a sufficient number of weak learners and they are sufficiently diverse
- to prove this, suppose each base model is a binary classifier with an accuracy of a, and suppose class 2 is the correct class
- let $y_e \in \{1, 2\}$ be the prediction for the eth model, and let $s = \sum_{e=1}^{E} y_e$ be the number of votes for class 2, where E is the number of models
- we define the final predictor to be the majority vote, i.e., class 2 if $s > E/2$ and class 1 otherwise

3.2 Voting

- the probability that the ensemble will pick class 2 is

$$p = p(s > E/2) = 1 - B(E/2, E, a),$$

  where $B(x, E, a)$ is the cumulative distribution function of the binomial distribution with parameters E and evaluated at x
- for $a = 0.51$ and $E = 1,000$, we get $p = 0.73$ and with $E = 10,000$, we get $p = 0.97$
- this means that, for a base model that is only slightly better than random guessing (51% accuracy), an ensemble with $1,000$ models has 73% accuracy, and an ensemble with $10,000$ models has 97% accuracy
- the performance of the voting approach is dramatically improved, because we assumed each predictor made independent errors
- this is clearly not the case, because they are trained on the same data
- they are likely to make the same types of errors, so there will be many majority votes for the wrong class, reducing the ensemble's accuracy

3.2 Voting

- ensemble methods work best when the predictors are as independent from one another as possible
- one way to get diverse classifiers is to train them using very different algorithms
- this increases the chance that they will make very different types of errors, improving the ensemble's accuracy
- if all classifiers are able to estimate class probabilities, then we can predict the class with the highest class probability, averaged over all the individual classifiers
- this is called *soft voting*; it often achieves higher performance than hard voting, because it gives more weight to highly confident votes

3.3 Bagging and pasting

- one way to get a diverse set of classifiers is to use very different training algorithms, as discussed
- another approach is to use the same training algorithm for every predictor and train them on different random subsets of the training set
- when sampling is performed *with* replacement, this method is called *bagging* (short for *bootstrap aggregating*, because resampling with replacement is called *bootstrapping* in statistics)
- when sampling is performed *without* replacement, it is called *pasting*
- in other words, both bagging and pasting allow training instances to be sampled several times across multiple predictors, but only bagging allows training instances to be sampled several times for the same predictor
- this sampling and training process is represented in Figure 9

3.3 Bagging and pasting

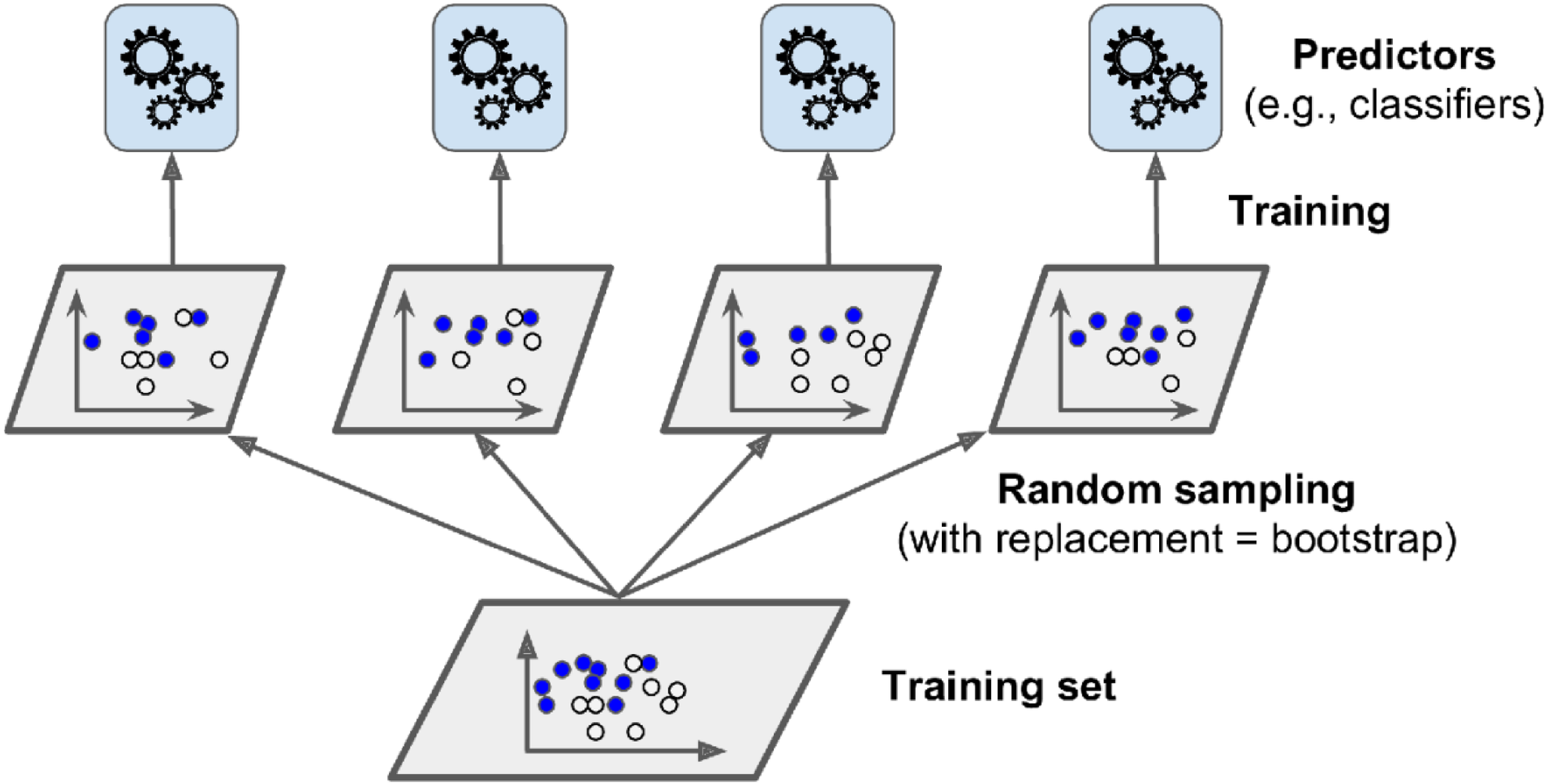


Figure 9: Bagging and pasting involves training several predictors on different random samples of the training set.

3.3 Bagging and pasting

- once all predictors are trained, the ensemble can make a prediction for a new instance by simply aggregating the predictions of all predictors
- the aggregation function is typically the *statistical mode* (i.e., the most frequent prediction, just like a hard voting classifier) for classification, or the *average* for regression
- each individual predictor has a higher bias than if it were trained on the original training set, but aggregation reduces both bias and variance
- generally, the net result is that the ensemble has a similar bias, but a lower variance than a single predictor trained on the original training set

3.3 Bagging and pasting

- decision trees suffer from *high variance*
- this means that if we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different
- in contrast, a procedure with *low variance* will yield similar results if applied repeatedly to distinct datasets; linear regression tends to have low variance, if the ratio of n to d is moderately large
- bagging is a general-purpose procedure for reducing the variance of a statistical learning method

3.3 Bagging and pasting

- it can be proved that, given a set of n independent observations $z_1, z_2, \ldots, z_n$, each with variance σ^2, the variance of the mean $\overline{z}$ of the observations is given by σ^2/n
- in other words, *averaging a set of observations reduces variance*
- hence, a natural way to reduce the variance and increase the test set accuracy of a statistical learning method is to take many training sets from the population, build a separate prediction model using each training set, and average the resulting predictions
- of course, this is not practical, because we generally do not have access to multiple training sets
- instead, we can bootstrap, by taking repeated samples from the (single) training dataset
- in this approach, we generate E different bootstrapped training datasets

3.3 Bagging and pasting

- while bagging can improve predictions for many regression methods, it is particularly useful for decision trees
- to apply bagging to regression trees, we simply construct E regression trees using E bootstrapped training sets, and average the resulting predictions
- these trees are grown deep, and are not pruned
- hence, each individual tree has high variance, but low bias
- averaging these E trees reduces the variance

3.3 Bagging and pasting

- bagging has been demonstrated to give impressive improvements in accuracy by combining together hundreds or even thousands of trees into a single procedure
- as we can see in Figure 9, predictors can all be trained in parallel, via different CPU cores or even different servers
- similarly, predictions can be made in parallel; this is one of the reasons bagging and pasting are such popular methods: they scale very well
- Figure 10 compares the decision boundary of a single decision tree with the decision boundary of a bagging ensemble of 500 trees, both trained on the `two_moons` dataset

3.3 Bagging and pasting

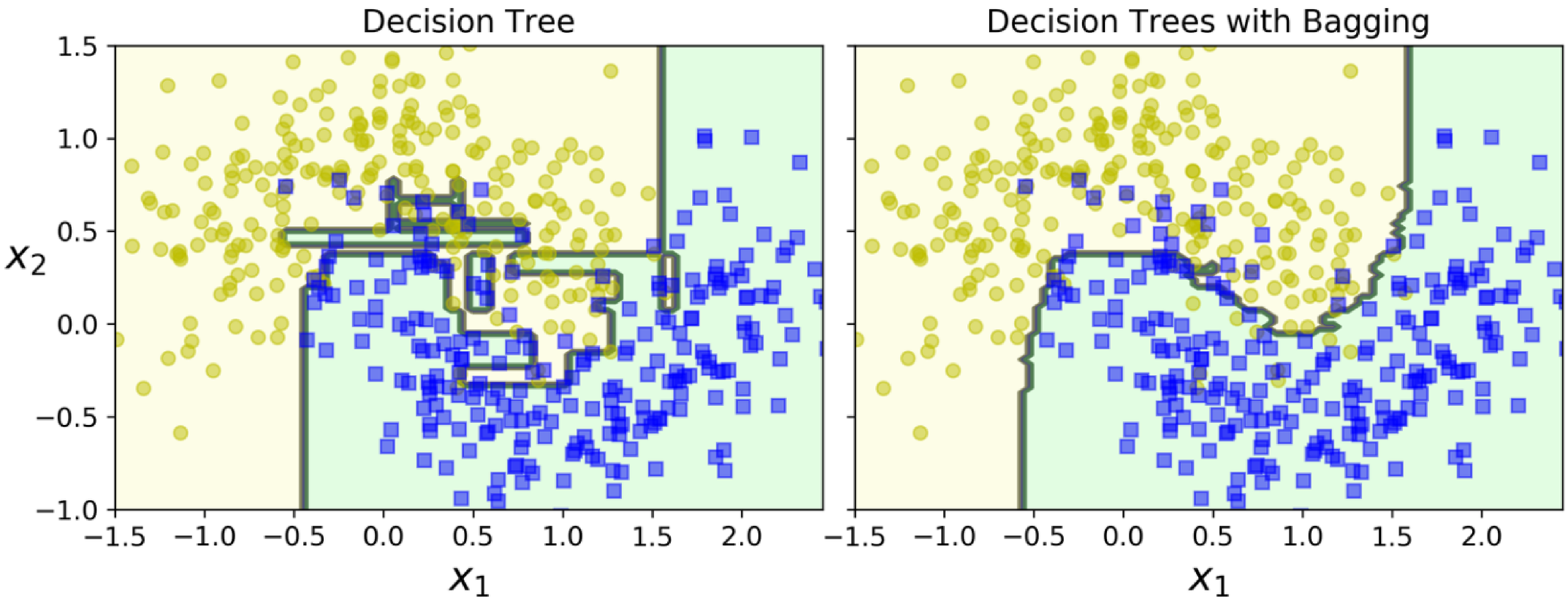


Figure 10: A single decision tree (left) versus a bagging ensemble of 500 trees (right).

3.3 Bagging and pasting

- as we can see, the ensemble's predictions will likely generalize much better than the single decision tree's predictions: the ensemble has a comparable bias but a smaller variance (it makes roughly the same number of errors on the training set, but the decision boundary is less irregular)
- the number of trees E is not a critical parameter with bagging; using a very large value of E will not lead to overfitting
- in practice, we use a value of E sufficiently large that the error has settled down
- bootstrapping introduces a bit more diversity in the subsets that each predictor is trained on, so bagging ends up with a slightly higher bias than pasting; but the extra diversity also means that the predictors end up being less correlated, so the ensemble's variance is reduced
- overall, bagging often results in better models, which explains why it is generally preferred

3.3 Bagging and pasting

- with bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all
- by default, a bagging classifier samples n training instances with replacement, where n is the size of the training set
- this means that only about 63% of the training instances are sampled on average for each predictor
- to see why, note that the chance that a single item will not be selected from a set of size n in any of n draws is $(1 - 1/n)^n$
- in the limit of large n, this becomes $\exp(-1) \approx 0.37$, which means only $1 - 0.37 = 0.63$ of the data points will be selected

3.3 Bagging and pasting

- the remaining 37% of the training instances, which are not sampled, are called *out-of-bag* (*oob*) instances
- note that they are not the same 37% for all predictors
- since a predictor never sees the oob instances during training, it can be evaluated on these instances, without the need for a separate validation set
- we can evaluate the ensemble itself by averaging out the oob evaluations of each predictor

3.3 Bagging and pasting

- for more diversity, we can sample the features as well; in this case, each predictor will be trained on a random subset of the input features
- this technique is particularly useful when we are dealing with high-dimensional inputs
- sampling both training instances and features is called the *random patches* method
- keeping all training instances but sampling features is called the *random subspaces* method
- sampling features results in even more predictor diversity, trading a bit more bias for a lower variance

3.3 Bagging and pasting

- the main advantage of bootstrap is that it prevents the ensemble from relying too much on any individual training example, which enhances robustness and generalization
- omitting a single example from the training set can have a large impact on the decision tree that we learn (even though the tree growing algorithm is otherwise deterministic)
- by averaging the predictions from both of these models, we get the more reasonable prediction model
- this advantage generally increases with the size of the ensemble

3.3 Bagging and pasting

- as we have discussed, bagging typically results in improved accuracy over prediction using a single tree
- unfortunately, however, it can be difficult to interpret the resulting model
- recall that one of the advantages of decision trees is the attractive and easily interpreted diagram that results
- however, when we bag a large number of trees, it is no longer possible to represent the resulting statistical learning procedure using a single tree, and it is no longer clear which variables are most important to the procedure
- thus, bagging improves prediction accuracy at the expense of interpretability

3.3 Bagging and pasting

- although the collection of bagged trees is much more difficult to interpret than a single tree, one can obtain an overall summary of the *importance* of each feature, using the mean squared error (for bagging regression trees) or the Gini index (for bagging classification trees)
- in the case of bagging regression trees, we can record the total amount that the mean squared error is decreased due to splits over a given feature, averaged over all E trees
- a large value indicates an important feature
- similarly, in the context of bagging classification trees, we can add up the total amount that the Gini index is decreased by splits over a given feature, averaged over all E trees
- thus, we obtain *feature importances*

3.4 Random forests

- a *random forest* is an ensemble of decision trees, generally trained via the bagging method (or sometimes pasting), typically with maximum number of samples set to the size of the training set
- the random forest algorithm introduces extra randomness when growing trees; instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features
- the algorithm results in greater tree diversity, which (again) trades a higher bias for a lower variance, generally yielding an overall better model

3.4 Random forests

- *random forests* provide an improvement over bagged trees by way of a small tweak that *decorrelates* the trees
- as in bagging, we build a number of decision trees on bootstrapped training samples
- but when building these decision trees, each time a split in a tree is considered, *a random sample of c features* is chosen as split candidates from the full set of *d* features
- the split is allowed to use only one of those *c* features
- a fresh sample of *c* features is taken at each split, and typically we choose $c \approx \sqrt{d}$ – that is, the number of features considered at each split is approximately equal to the square root of the total number of features

3.4 Random forests

- in other words, in building a random forest, at each split in the tree, the algorithm is *not even allowed to consider* a majority of the available features; this may sound crazy, but it has a clever rationale
- suppose that there is one very strong feature in the dataset, along with a number of other moderately strong features
- then, in the collection of bagged trees, most or all of the trees will use this strong feature in the top split
- consequently, all of the bagged trees will look quite similar to each other
- hence, the predictions from the bagged trees will be highly correlated

3.4 Random forests

- unfortunately, averaging many highly correlated quantities does not lead to as large of a reduction in variance as averaging many uncorrelated quantities
- in particular, this means that bagging will not lead to a substantial reduction in variance over a single tree, in this setting
- random forests overcome this problem by forcing each split to consider only a subset of the features
- therefore, on average, $(d - c)/d$ of the splits will not even consider the strong feature, and so other features will have more of a chance
- we can think of this process as *decorrelating* the trees, thereby making the average of the resulting trees less variable and hence more reliable

3.4 Random forests

- the main difference between bagging and random forests is the choice of feature subset size c
- for instance, if a random forest is built using $c = d$, then this amounts simply to bagging
- using a small value of c in building a random forest will typically be helpful when we have a large number of correlated features
- as with bagging, random forests will not overfit if we increase E, so in practice we use a value of E sufficiently large for the error rate to have settled down

3.4 Random forests

- when we are growing a tree in a random forest, at each node, only a random subset of the features is considered for splitting (as discussed earlier)
- it is possible to make trees even more random by also using random thresholds for each feature, rather than searching for the best possible thresholds (like regular decision trees do)
- a forest of such extremely random trees is called an *extremely randomized trees* ensemble (or *extra-trees* for short)
- once again, this technique trades more bias for a lower variance
- it also makes extra-trees much faster to train than regular random forests, because finding the best possible threshold for each feature at every node is one of the most time-consuming tasks of growing a tree

3.5 Boosting

- *boosting* (originally called *hypothesis boosting*) refers to any ensemble method that can combine several weak learners into a strong learner
- the general idea of most boosting methods is to train predictors sequentially, each trying to correct its predecessor
- there are many boosting methods available, but by far the most popular are *AdaBoost* (short for *adaptive boosting*) and *gradient boosting*
- boosting works in a similar way as bagging, except that the trees are grown *sequentially*: each tree is grown using information from previously grown trees
- boosting does not involve bootstrap sampling; instead, each tree is fit on a modified version of the original dataset

3.5 Boosting

- unlike fitting a single large decision tree to the data, which amounts to *fitting the data hard* and potentially overfitting, the boosting approach instead *learns slowly*
- given the current model, we fit a decision tree to the *residuals* from the model
- that is, we fit a tree using the current residuals, rather than the outcome, as the response
- we then add this new decision tree into the fitted function in order to update the residuals

3.5 Boosting

- each of these trees can be rather small, with just a few terminal nodes, determined by a parameter in the algorithm
- by fitting small trees to the residuals, we slowly improve the ensemble prediction in areas where it does not perform well
- the shrinkage parameter slows the process down even further, allowing more and different shaped trees to attack the residuals
- in general, statistical learning approaches that *learn slowly* tend to perform well
- note that in boosting, unlike in bagging, the construction of each tree depends strongly on the trees that have already been grown

3.5 Boosting

- one way for a new predictor to correct its predecessor is to pay a bit more attention to the training instances that the predecessor underfitted
- this results in new predictors focusing more and more on the hard cases; this is the technique used by AdaBoost
- when training an AdaBoost classifier, the algorithm first trains a base classifier (such as a decision tree) and uses it to make predictions on the training set
- the algorithm then increases the relative weight of misclassified training instances
- then it trains a second classifier, using the updated weights, and again makes predictions on the training set, updates the instance weights, and so on (see Figure 11)

3.5 Boosting

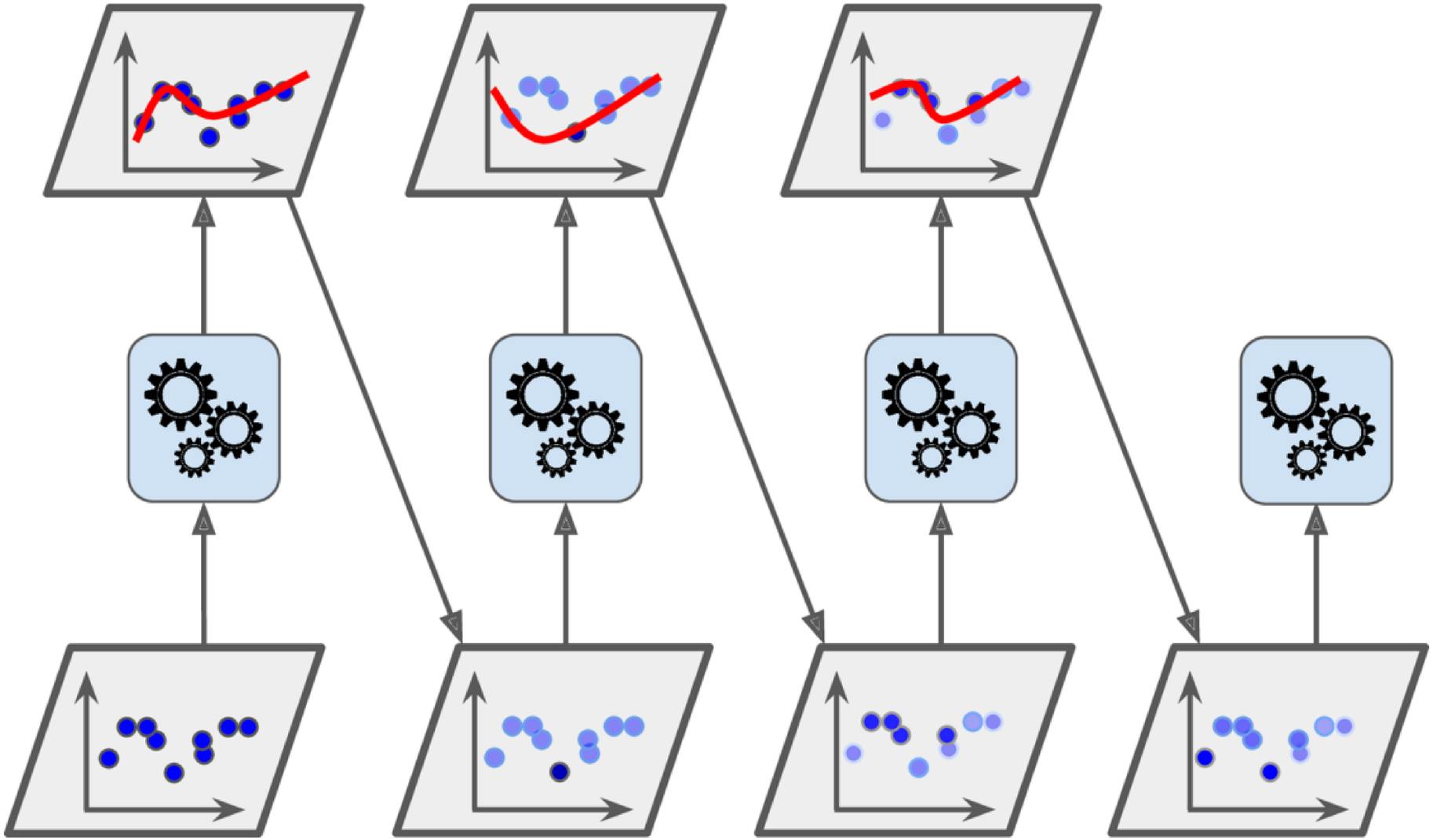


Figure 11: AdaBoost sequential training with instance weight updates.

- Figure 12 shows the decision boundaries of five consecutive predictors on the `two_moons` dataset

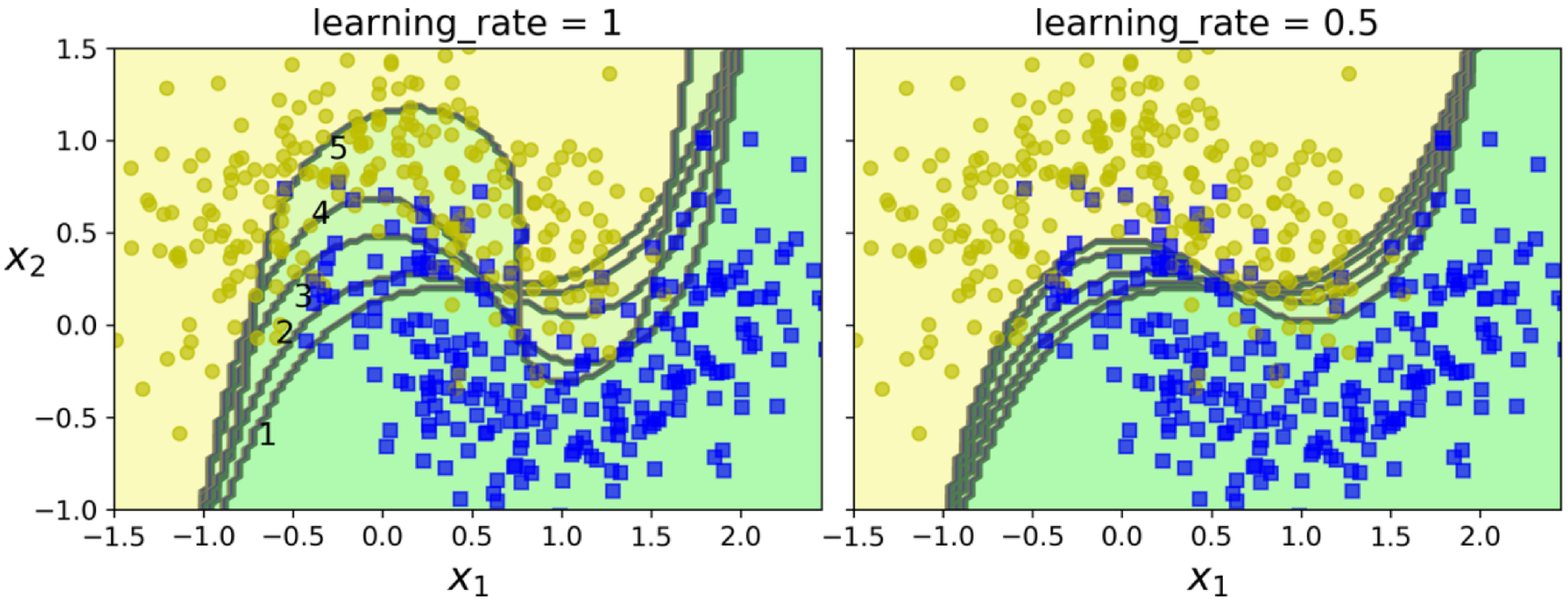


Figure 12: Decision boundaries of consecutive predictors.

3.5 Boosting

- in this example, each predictor is a highly regularized SVM classifier with an Gaussian RBF kernel, just for illustrative purposes
- the first classifier gets many instances wrong, so their weights get boosted
- the second classifier therefore does a better job on these instances, and so on
- the plot on the right represents the same sequence of predictors, except that the learning rate is halved (i.e., the misclassified instance weights are boosted half as much at every iteration)
- as we can see, this sequential learning technique has some similarities with gradient descent, except that, instead of tweaking a single predictor's parameters to minimize a loss function, AdaBoost adds predictors to the ensemble, gradually making it better

3.5 Boosting

- once all predictors are trained, the ensemble makes predictions very much like bagging or pasting, except that predictors have different weights depending on their overall accuracy on the weighted training set
- there is one important drawback to this sequential learning technique: it cannot be parallelized (or only partially), since each predictor can only be trained after the previous predictor has been trained and evaluated
- as a result, it does not scale as well as bagging or pasting

3.5 Boosting

- let's take a closer look at the AdaBoost algorithm
- each instance weight $w^{(i)}$ is initially set to $1/E$
- the eth predictor is trained, and its weighted error rate r_e is computed on the training set, using the following formula:

$$r_e = \frac{\sum_{\substack{i=1 \\ \hat{y}_e^{(i)} \neq y^{(i)}}}^{n} w_e^{(i)}}{\sum_{i=1}^{n} w_e^{(i)}},$$

where $\hat{y}_e^{(i)}$ is the eth predictor's prediction for the ith instance

3.5 Boosting

- the predictor's weight α_e is then computed using:

$$\alpha_e = \eta \log \frac{1 - r_e}{r_e},$$

  where η is the learning rate hyperparameter (defaults to 1)
- the more accurate the predictor is, the higher its weight will be; if it is just guessing randomly, then its weight will be close to zero
- however, if it is most often wrong (i.e., less accurate than random guessing), then its weight will be negative
- next, the AdaBoost algorithm updates the instance weights, using:

$$w_{e+1}^{(i)} \leftarrow \begin{cases} w_e^{(i)}, & \text{if } \hat{y}_e^{(i)} = y^{(i)} \\ w_e^{(i)} \exp(\alpha_e), & \text{if } \hat{y}_e^{(i)} \neq y^{(i)} \end{cases}, \text{ for } i = 1, 2, \ldots, n,$$

  which boosts the weights of the misclassified instances
- then all the instance weights are normalized (i.e., divided by $\sum_{i=1}^{n} w_{e+1}^{(i)}$)

3.5 Boosting

- finally, a new predictor $e + 1$ is trained using the updated weights, and the whole process is repeated (the new predictor's weight is computed, the instance weights are updated, then another predictor is trained, and so on)
- the algorithm stops when the desired number of predictors is reached, or when a perfect predictor is found
- to make predictions, AdaBoost simply computes the predictions of all the predictors and weighs them using the predictor weights α_e
- the predicted class is the one that receives the majority of weighted votes:

$$\hat{y}(\boldsymbol{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{e=1 \\ \hat{y}_e(\boldsymbol{x})=k}}^{E} \alpha_e,$$

where E is the number of predictors and $k = 1, 2, \ldots, q$ is a class

3.5 Boosting

- if the AdaBoost ensemble is overfitting the training set, we can try reducing the number of estimators or more strongly regularizing the base estimator
- another very popular boosting algorithm is *gradient boosting*
- just like AdaBoost, gradient boosting works by sequentially adding predictors to an ensemble, each one correcting its predecessor
- however, instead of tweaking the instance weights at every iteration like AdaBoost does, this method tries to fit the new predictor to the residual errors made by the previous predictor

3.5 Boosting

- let's go through a simple regression example, using decision trees as the base predictors (of course, gradient boosting also works great with classification tasks)
- this is called *gradient tree boosting*, or *gradient boosted regression trees* (*GBRT*)
- first, let's fit a decision tree regressor to the training set (for example, a noisy quadratic training set):

$$\hat{y}_1 = h_1(x_1, y).$$

3.5 Boosting

- next, we will train a second decision tree regressor on the residual errors made by the first predictor:

$$y_2 = y - \hat{y}_1,$$

$$\hat{y}_2 = h_2(x_1, y_2).$$

- then we train a third regressor on the residual errors made by the second predictor:

$$y_3 = y_2 - \hat{y}_2,$$

$$\hat{y}_3 = h_3(x_1, y_3).$$

- now we have an ensemble containing three trees
- it can make predictions on a new instance simply by adding up the predictions of all the trees:

$$\hat{y} = \hat{y}_1 + \hat{y}_2 + \hat{y}_3.$$

- Figure 13 represents the predictions of these three trees in the left column, and the ensemble's predictions in the right column

3.5 Boosting

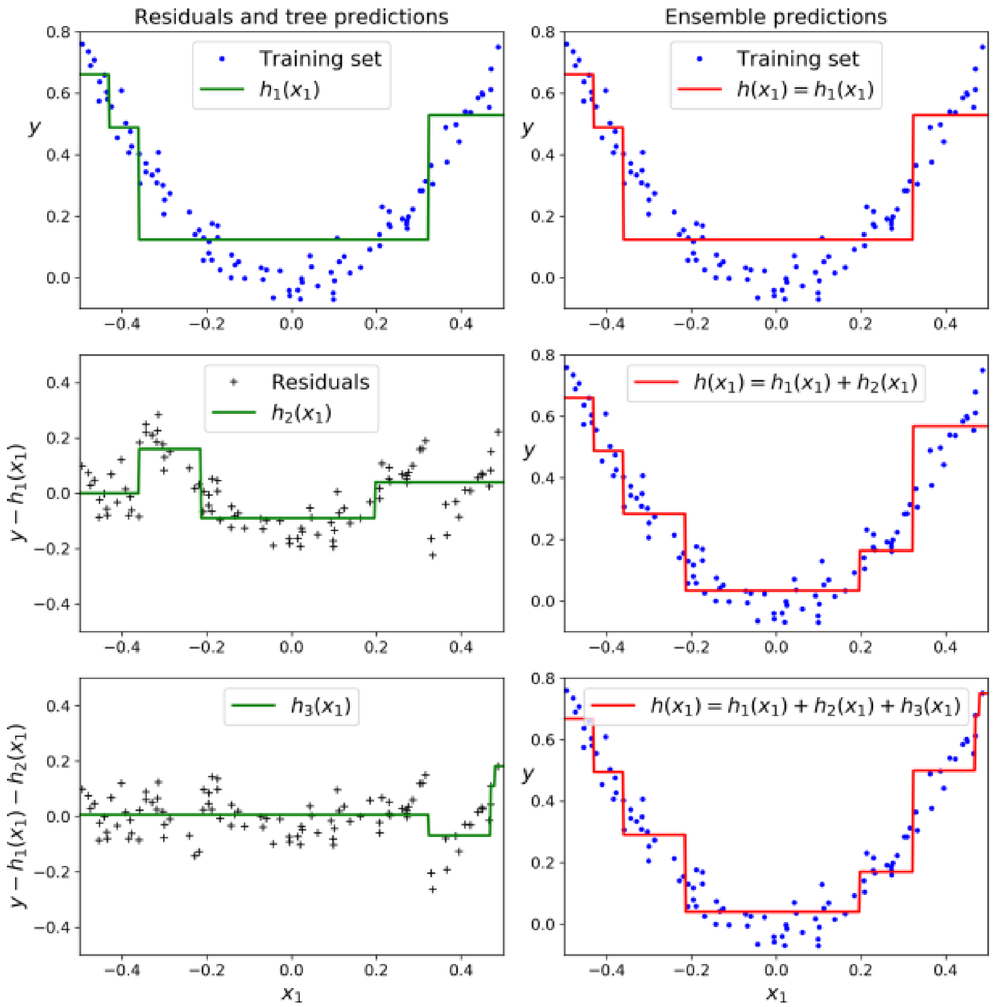


Figure 13: In this depiction of gradient boosting, the first predictor (top left) is trained normally, then each consecutive predictor (middle left and lower left) is trained on the previous predictor's residuals; the right column shows the resulting ensemble's predictions.

3.5 Boosting

- in the first row, the ensemble has just one tree, so its predictions are exactly the same as the first tree's predictions
- in the second row, a new tree is trained on the residual errors of the first tree
- on the right, we can see that the ensemble's predictions are equal to the sum of the predictions of the first two trees
- similarly, in the third row, another tree is trained on the residual errors of the second tree
- we can see that the ensemble's predictions gradually get better as trees are added to the ensemble

3.5 Boosting

- the *learning rate* hyperparameter scales the contribution of each tree
- if we set it to a low value, such as 0.1, we will need more trees in the ensemble to fit the training set, but the predictions will usually generalize better
- this is a regularization technique called *shrinkage*
- Figure 14 shows two GBRT ensembles trained with a low learning rate: the one on the left does not have enough trees to fit the training set, while the one on the right has too many trees and overfits the training set

3.5 Boosting

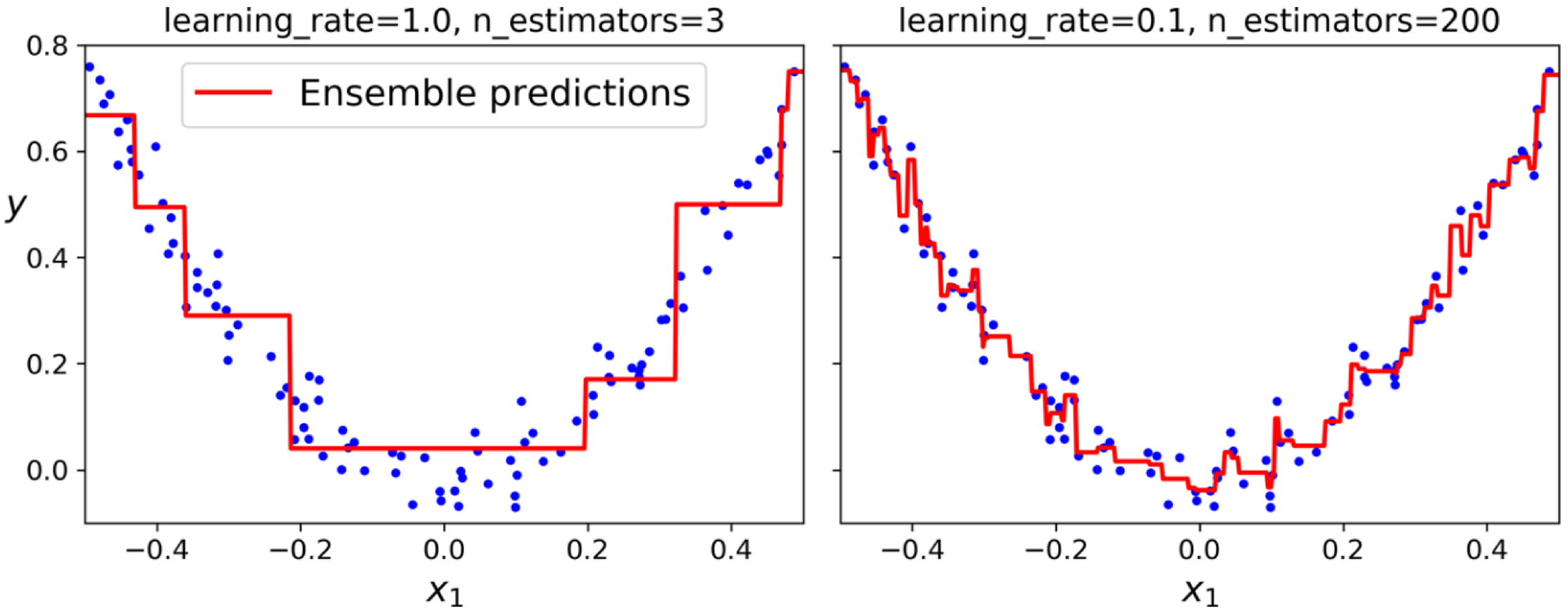


Figure 14: GBRT ensembles with not enough predictors (left) and too many (right).

3.5 Boosting

- we can also specify the fraction of training instances to be used for training each tree
- for example, each tree can be trained on 25% of the training instances, selected randomly
- this technique trades a higher bias for a lower variance; it also speeds up training considerably
- this is called *stochastic gradient boosting*

3.6 Stacking

- the last ensemble method we will discuss in this lecture is called *stacking* (short for *stacked generalization*)
- it is based on a simple idea: instead of using trivial functions (such as hard voting) to aggregate the predictions of all predictors in an ensemble, why don't we train a model to perform this aggregation?
- Figure 15 shows such an ensemble performing a regression task on a new instance
- each of the bottom three predictors predicts a different value (3.1, 2.7, and 2.9), and then the final predictor (called a *blender*, or a *meta learner*) takes these predictions as inputs and makes the final prediction (3.0)

3.6 Stacking

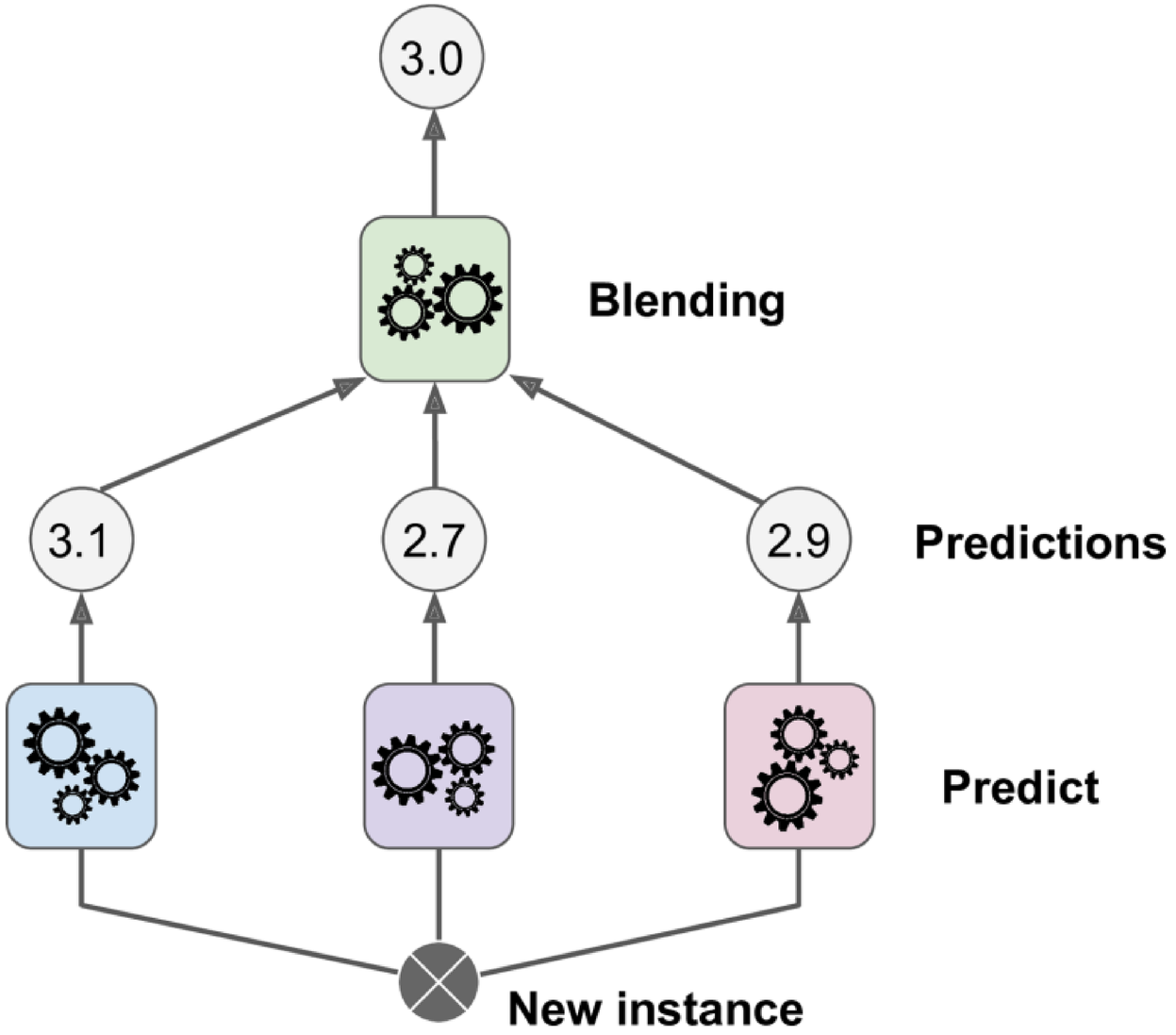


Figure 15: Aggregating predictions using a blending predictor.

3.6 Stacking

- to train the blender, a common approach is to use a hold-out set
- alternatively, it is possible to use out-of-fold predictions
- in some contexts, this is called *stacking*, while using a hold-out set is called *blending*
- let's see how using a hold-out set works; first, the training set is split into two subsets
- the first subset is used to train the predictors in the first layer (see Figure 16)

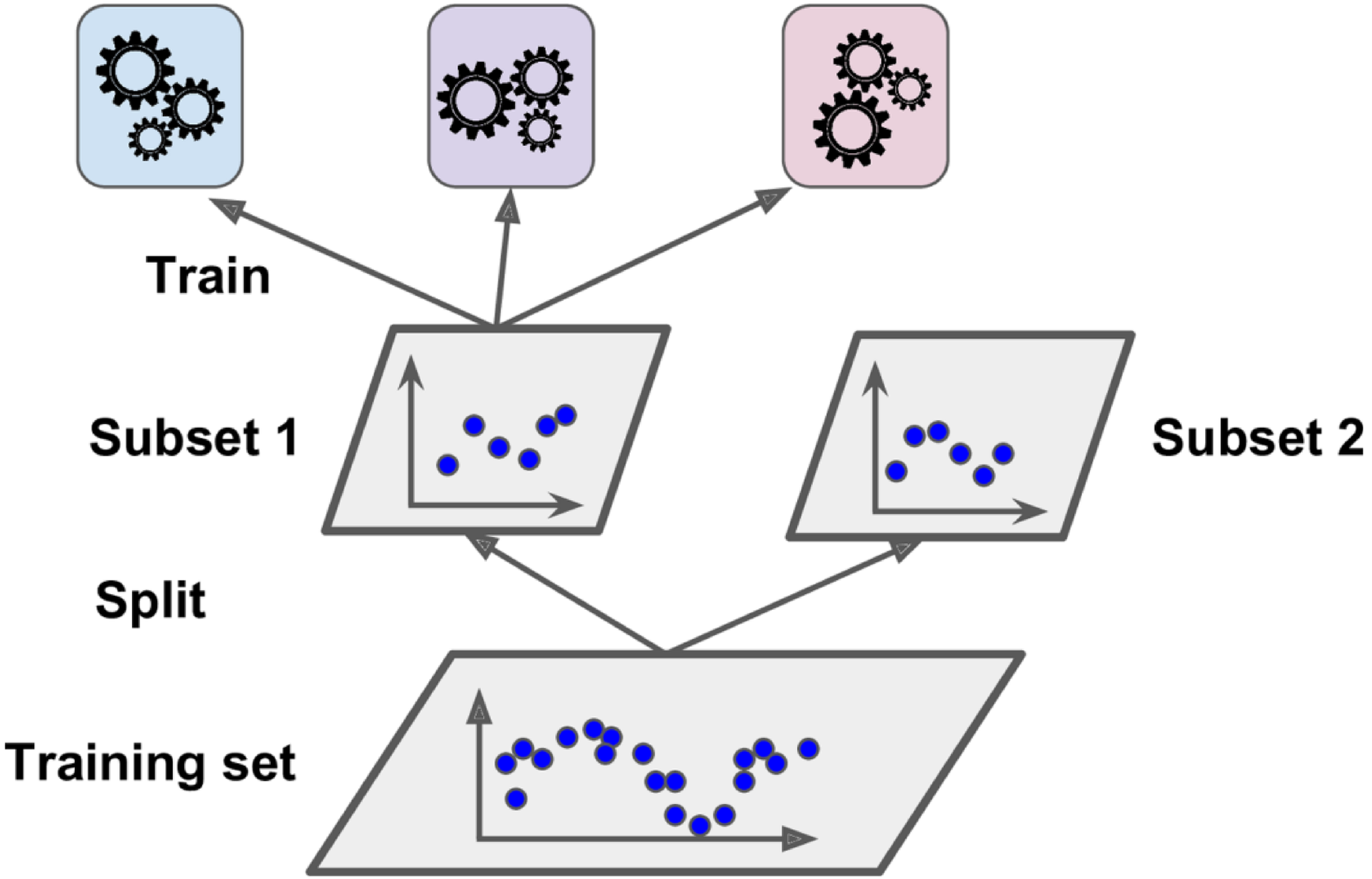


Figure 16: Training the first layer.

3.6 Stacking

- next, the first layer's predictors are used to make predictions on the second (held-out) set (see Figure 17)
- this ensures that the predictions are "clean", since the predictors never saw these instances during training
- for each instance in the hold-out set, there are three predicted values
- we can create a new training set using these predicted values as input features (which makes this new training set 3D), and keeping the target values
- the blender is trained on this new training set, so it learns to predict the target value, given the first layer's predictions

3.6 Stacking

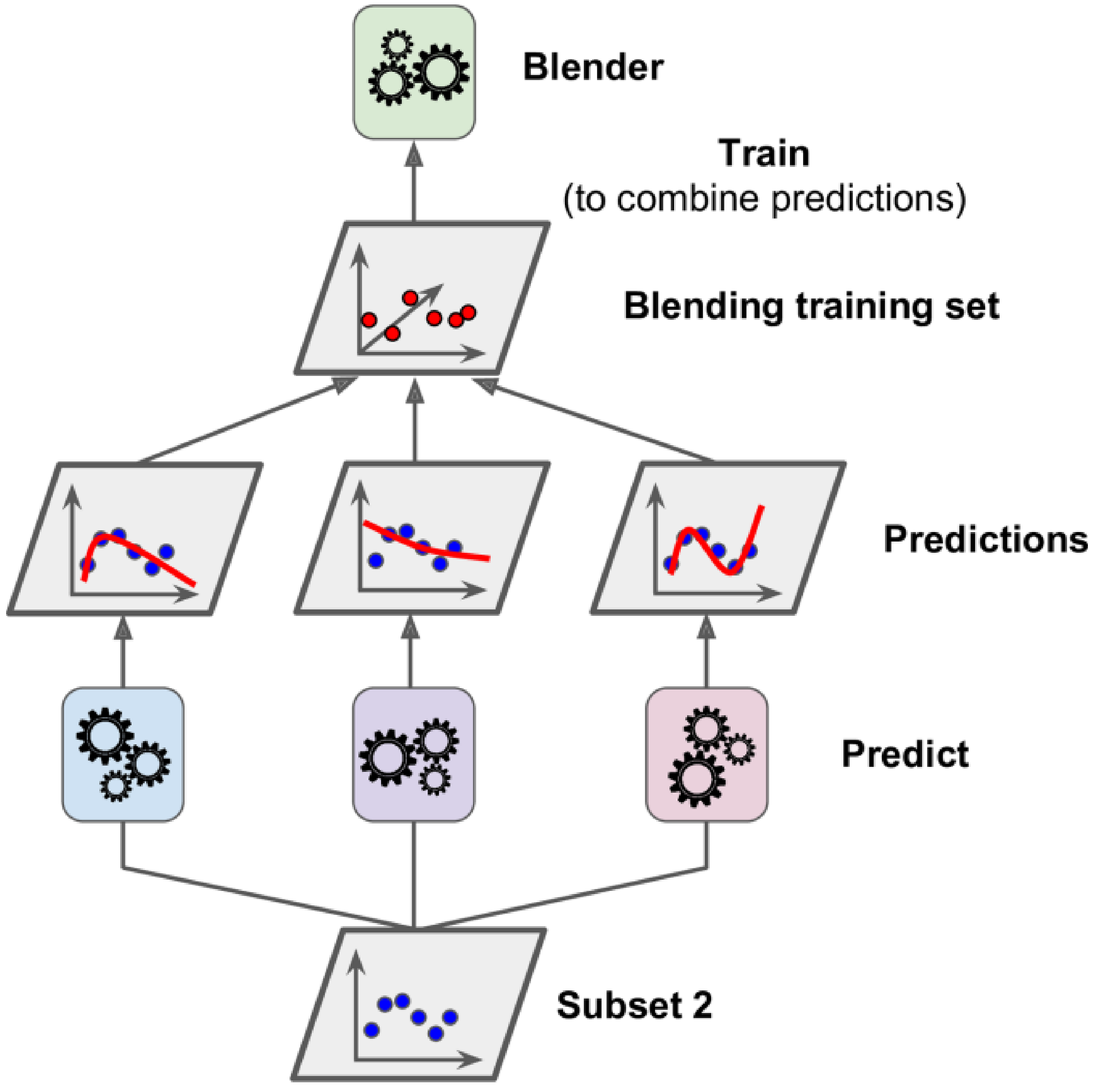


Figure 17: Training the blender.

3.6 Stacking

- it is actually possible to train several different blenders this way (e.g., one using linear regression, another using random forest regression), to get a whole layer of blenders
- the trick is to split the training set into three subsets: the first one is used to train the first layer, the second one is used to create the training set used to train the second layer (using predictions made by the predictors of the first layer), and the third one is used to create the training set to train the third layer (using predictions made by the predictors of the second layer)
- once this is done, we can make a prediction for a new instance by going through each layer sequentially, as shown in Figure 18

3.6 Stacking

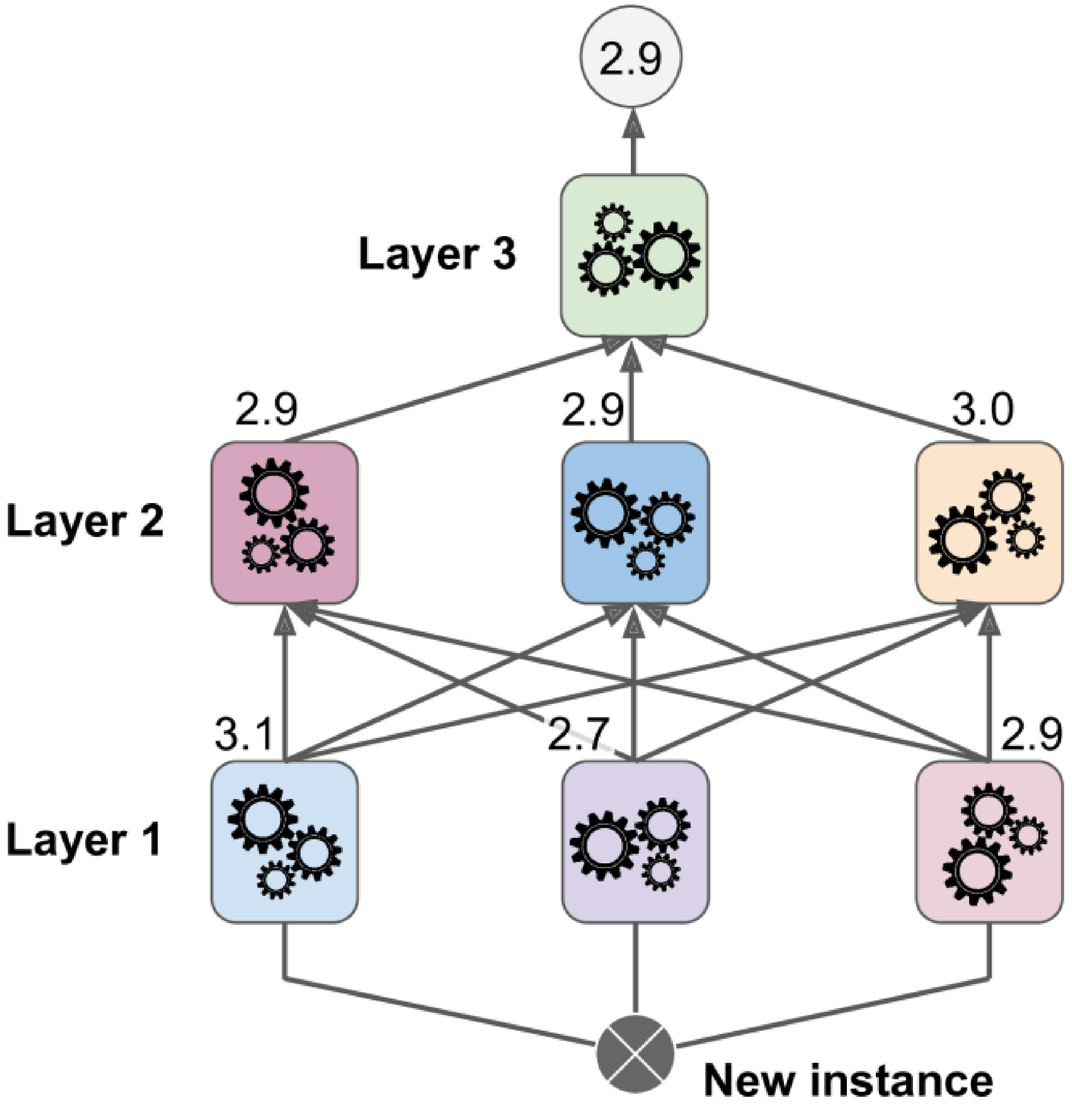


Figure 18: Predictions in a multilayer stacking ensemble.

Exercises

Exercise 3

Consider training a classification decision tree given a design matrix $\boldsymbol{X} = \begin{bmatrix} 6 & 3 \\ 2 & 7 \\ 9 & 6 \\ 4 & 2 \end{bmatrix}$ and labels $\boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$. Let x_1 denote feature 1, corresponding to the first column of $\boldsymbol{X}$, and let x_2 denote feature 2, corresponding to the second column of X. Which of the following splits at the root node gives the smallest Gini index: $x_1 > 2$, $x_1 > 4$, $x_2 > 3$, $x_2 > 6$? Motivate your answer.

Exercises

Solution:

- the *Gini index* is defined by:

$$G_h = \sum_{k=1}^{q} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \sum_{k=1}^{q} \hat{p}_{hk} - \sum_{k=1}^{q} \hat{p}_{hk}^2 = 1 - \sum_{k=1}^{q} \hat{p}_{hk}^2,$$

where $\hat{p}_{hk}$ represents the proportion of training examples in the hth region that are from the kth class:

$$\hat{p}_{hk} = \frac{1}{|\mathcal{D}_h|} \sum_{i:\,(\boldsymbol{x}^{(i)}, y^{(i)}) \in \mathcal{D}_h} 1_{y^{(i)}=k}.$$

Exercises

- we have $q = 2$
- for $x_1 > 2$ we have $\left(\begin{bmatrix} 6 & 3 \end{bmatrix}^\top, 1\right)$, $\left(\begin{bmatrix} 9 & 6 \end{bmatrix}^\top, 1\right)$, $\left(\begin{bmatrix} 4 & 2 \end{bmatrix}^\top, 0\right)$ and $\left(\begin{bmatrix} 2 & 7 \end{bmatrix}^\top, 0\right)$
- the Gini index is:

$$G_1 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{1}{3}\left(1 - \frac{1}{3}\right) + \frac{2}{3}\left(1 - \frac{2}{3}\right) = \frac{4}{9}$$

$$G_2 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{1}{1}\left(1 - \frac{1}{1}\right) + \frac{0}{1}\left(1 - \frac{0}{1}\right) = 0$$

Exercises

- for $x_1 > 4$ we have $\left(\begin{bmatrix} 6 & 3 \end{bmatrix}^\top, 1\right)$, $\left(\begin{bmatrix} 9 & 6 \end{bmatrix}^\top, 1\right)$ and $\left(\begin{bmatrix} 2 & 7 \end{bmatrix}^\top, 0\right)$, $\left(\begin{bmatrix} 4 & 2 \end{bmatrix}^\top, 0\right)$
- the Gini index is:

$$G_1 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{0}{2}\left(1 - \frac{0}{2}\right) + \frac{2}{2}\left(1 - \frac{2}{2}\right) = 0$$

$$G_2 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{2}{2}\left(1 - \frac{2}{2}\right) + \frac{0}{2}\left(1 - \frac{0}{2}\right) = 0$$

Exercises

- for $x_2 > 3$ we have $\left(\begin{bmatrix} 2 & 7 \end{bmatrix}^\top, 0\right)$, $\left(\begin{bmatrix} 9 & 6 \end{bmatrix}^\top, 1\right)$ and $\left(\begin{bmatrix} 6 & 3 \end{bmatrix}^\top, 1\right)$, $\left(\begin{bmatrix} 4 & 2 \end{bmatrix}^\top, 0\right)$
- the Gini index is:

$$G_1 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{1}{2}\left(1 - \frac{1}{2}\right) + \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{2}$$

$$G_2 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{1}{2}\left(1 - \frac{1}{2}\right) + \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{2}$$

Exercises

- for $x_2 > 6$ we have $\left(\begin{bmatrix} 2 & 7 \end{bmatrix}^\top, 0\right)$ and $\left(\begin{bmatrix} 6 & 3 \end{bmatrix}^\top, 1\right)$, $\left(\begin{bmatrix} 9 & 6 \end{bmatrix}^\top, 1\right)$, $\left(\begin{bmatrix} 4 & 2 \end{bmatrix}^\top, 0\right)$
- the Gini index is:

$$G_1 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{1}{1}\left(1 - \frac{1}{1}\right) + \frac{0}{1}\left(1 - \frac{0}{1}\right) = 0$$

$$G_2 = \sum_{k=1}^{2} \hat{p}_{hk}(1 - \hat{p}_{hk}) = \frac{1}{3}\left(1 - \frac{1}{3}\right) + \frac{2}{3}\left(1 - \frac{2}{3}\right) = \frac{4}{9}$$

- thus, the smallest Gini index is given by $x_1 > 4$

Vă mulţumesc!